Lumentra Admin Guide

Lumentra AI LLC

March 2026

Contents

Lumentra Administration Guide **4**
- Table of Contents . . . 4
- 1. System Overview . . . 4
  - What Lumentra Does . . . 5
  - Supported Industries . . . 5
  - Industry-Specific Pipeline Stages . . . 5
- 2. Architecture . . . 5
  - Component Overview . . . 5
  - Service Stack . . . 6
  - Voice Pipeline . . . 6
  - API Route Structure . . . 7
- 3. Dashboard Walkthrough . . . 7
  - Dashboard Layout . . . 7
  - 3.1 Dashboard Home (`/dashboard`) . . . 7
  - 3.2 Calls Page (`/calls`) . . . 8
  - 3.3 Contacts Page (`/contacts`) . . . 8
  - 3.4 Calendar Page (`/calendar`) . . . 8
  - 3.5 Deals Page (`/deals`) . . . 8
  - 3.6 Tasks Page (`/tasks`) . . . 9
  - 3.7 Escalations Page (`/escalations`) . . . 9
  - 3.8 Pending Bookings Page (`/pending`) . . . 9
  - 3.9 Resources Page (`/resources`) . . . 9
  - 3.10 Notifications Page (`/notifications`) . . . 10
  - 3.11 Analytics Page (`/analytics`) . . . 10
  - 3.12 Workstation Page (`/workstation`) . . . 10
  - 3.13 Settings Pages (`/settings`) . . . 10
  - 3.14 Profile Page (`/profile`) . . . 11
  - 3.15 Setup Wizard (`/setup`) . . . 12
- 4. Tenant Management . . . 12
  - Understanding Tenants . . . 12
  - Tenant Lifecycle . . . 12
  - Creating a Tenant via API . . . 12
  - Updating a Tenant . . . 13
  - Listing Tenants . . . 13
  - Phone Number Updates . . . 13
- 5. User and Role Management . . . 14
  - Authentication . . . 14
  - Roles . . . 14
  - Role Enforcement . . . 14
  - Member Management . . . 14
- 6. System Configuration . . . 15
  - Voice Pipeline Configuration . . . 15
  - Voice Agent Settings . . . 15

Operating Hours . . . 15
Feature Flags . . . 15
Max Call Duration . . . 16
7. API Endpoint Reference . . . 16
7.1 Health Routes (`/health`) . . . 16
7.2 Chat Routes (`/api/chat`) . . . 16
7.3 Internal Routes (`/internal`) . . . 17
7.4 Tenant Routes (`/api/tenants`) . . . 19
7.5 Call Routes (`/api/calls`) . . . 19
7.6 Contact Routes (`/api/contacts`) . . . 20
7.7 Booking Routes (`/api/bookings`) . . . 21
7.8 Availability Routes (`/api/availability`) . . . 22
7.9 Resource Routes (`/api/resources`) . . . 22
7.10 Escalation Routes (`/api/escalation`) . . . 23
7.11 Notification Routes (`/api/notifications`) . . . 23
7.12 Dashboard Routes (`/api/dashboard`) . . . 24
7.13 Deal Routes (`/api/deals`) . . . 24
7.14 Task Routes (`/api/tasks`) . . . 26
7.15 Voicemail Routes (`/api/voicemails`) . . . 27
7.16 Promotion Routes (`/api/promotions`) . . . 27
7.17 Capability Routes (`/api/capabilities`) . . . 27
7.18 Training Data Routes (`/api/training-data`) . . . 28
7.19 Pending Booking Routes (`/api/pending-bookings`) . . . 28
7.20 Integration Routes (`/api/integrations`) . . . 29
7.21 Setup Wizard Routes (`/api/setup`) . . . 29
7.22 Phone Configuration Routes (`/api/phone`) . . . 29
8. Database Table Reference . . . 30
8.1 tenants . . . 30
8.2 calls . . . 31
8.3 bookings . . . 32
8.4 contacts . . . 32
8.5 contact_notes . . . 33
8.6 contact_activity . . . 34
8.7 resources . . . 34
8.8 availability_templates . . . 34
8.9 availability_slots . . . 35
8.10 notification_templates . . . 35
8.11 notifications . . . 35
8.12 notification_preferences . . . 36
8.13 audit_logs . . . 36
8.14 tenant_members . . . 36
8.15 voicemails . . . 37
8.16 conversation_logs . . . 37
8.17 callback_queue . . . 38
8.18 sms_messages . . . 39
8.19 deals . . . 39
8.20 tasks . . . 39
8.21 phone_configurations . . . 40
8.22 port_requests . . . 40
8.23 Additional Tables . . . 41
9. Configuration Deep-Dive . . . 41
9.1 API Environment Variables (`lumentra-api/.env.example`) . . . 41
9.2 Dashboard Environment Variables (`lumentra-dashboard/.env.example`) . . . 42
9.3 Agent Environment Variables (`lumentra-agent/.env.example`) . . . 42
9.4 Encryption Configuration . . . 43
10. Background Jobs Reference . . . 43
10.1 Reminder Processing . . . 43

10.2 Callback Processing . . . 43
10.3 Notification Queue Processing . . . 44
10.4 Engagement Score Calculation . . . 44
10.5 Availability Slot Generation . . . 44
10.6 Review Request Sending . . . 44
11. Webhook and Integration Reference . . . 45
11.1 SignalWire SIP Webhook . . . 45
11.2 LiveKit Agent Communication . . . 45
11.3 Twilio Webhooks . . . 45
11.4 OAuth Integration Flow . . . 45
11.5 Post-Call Automation . . . 46
12. Permission Matrix . . . 46
12.1 Tenant Management . . . 46
12.2 CRM Data (Calls, Contacts, Bookings) . . . 46
12.3 Pipeline (Deals & Tasks) . . . 47
12.4 Escalation . . . 47
12.5 Settings & Configuration . . . 48
12.6 Rate Limits . . . 48
13. Audit Trail Documentation . . . 49
13.1 Audit Log Table . . . 49
13.2 Audited Actions . . . 49
13.3 Querying Audit Logs . . . 49
14. Multi-Tenancy Deep-Dive . . . 50
14.1 Database Level – Row Level Security (RLS) . . . 50
14.2 Application Cache Level . . . 50
14.3 API Middleware Level . . . 51
14.4 Cross-Tenant Data Isolation . . . 51
15. Notification Templates . . . 51
15.1 Built-in Templates . . . 51
15.2 Template Rendering . . . 52
15.3 Notification Channels . . . 52
15.4 Notification Preferences . . . 52
16. Managing Voice Agents . . . 53
16.1 Voice Agent Architecture . . . 53
16.2 Voice Tool Functions . . . 53
16.3 Tool Input Validation . . . 53
16.4 Greeting Configuration . . . 53
17. Phone Number Management . . . 54
17.1 Phone Setup Types . . . 54
17.2 Number Porting Process . . . 54
17.3 Call Forwarding Setup . . . 54
17.4 SIP Configuration . . . 54
18. Escalation Management . . . 54
18.1 Escalation Triggers . . . 54
18.2 Escalation Flow . . . 55
18.3 Escalation Queue . . . 55
18.4 Escalation Contacts . . . 55
19. CRM and Automation . . . 55
19.1 Contact Lifecycle . . . 55
19.2 Deal Pipeline . . . 55
19.3 Task Management . . . 55
19.4 Contact Import and Export . . . 56
20. Monitoring and Health . . . 56
20.1 Health Endpoints . . . 56
20.2 Dashboard Metrics . . . 56
20.3 Key Metrics to Monitor . . . 56
21. Server and Container Management . . . 56

21.1 Deployment Architecture . . . 56
21.2 Container Management . . . 57
21.3 Port Requirements . . . 57
22. Security . . . 57
22.1 Authentication Layers . . . 57
22.2 Data Protection . . . 57
22.3 Sensitive Data Handling . . . 57
22.4 Rate Limiting Details . . . 58
23. Troubleshooting . . . 58
23.1 Voice Pipeline Issues . . . 58
23.2 Authentication Issues . . . 58
23.3 Database Issues . . . 59
23.4 Notification Issues . . . 59
23.5 Cache Issues . . . 59
23.6 Integration Issues . . . 59
23.7 Background Job Issues . . . 60

Lumentra Administration Guide

Version: 2.0 **Last Updated:** 2026-03-02 **Audience:** System administrators, platform operators, tenant owners, and developers

---

Table of Contents

1. System Overview
2. Architecture
3. Dashboard Walkthrough
4. Tenant Management
5. User and Role Management
6. System Configuration
7. API Endpoint Reference
8. Database Table Reference
9. Configuration Deep-Dive
10. Background Jobs Reference
11. Webhook and Integration Reference
12. Permission Matrix
13. Audit Trail Documentation
14. Multi-Tenancy Deep-Dive
15. Notification Templates
16. Managing Voice Agents
17. Phone Number Management
18. Escalation Management
19. CRM and Automation
20. Monitoring and Health
21. Server and Container Management
22. Security
23. Troubleshooting

---

1. System Overview

Lumentra is a multi-tenant voice AI platform that answers inbound phone calls for businesses 24/7. Each tenant represents a business with its own phone number, voice agent personality, operating

hours, and CRM data. The platform handles the full lifecycle from incoming call through conversation, booking, follow-up automation, and analytics.

What Lumentra Does

- Answers inbound phone calls with an AI voice agent tailored to each business
- Handles reservations, appointments, FAQs, message-taking, and order capture
- Automatically creates contacts, logs calls, and generates transcripts
- Escalates to human staff when needed (live transfer or callback queue)
- Sends SMS confirmations and reminders via Twilio
- Provides a dashboard with call analytics, CRM pipeline, and task management
- Runs post-call automation to create deals, tasks, and update contact statuses
- Supports multiple industries with tailored pipelines, terminology, and capabilities

Supported Industries

Each industry gets tailored capabilities, prompts, pipeline stages, and terminology:

Industry	Pipeline	Capabilities	Task Types
`restaurant`	Reservations	Reservations, takeaway, menu	Vendor Order, Event Setup
`pizza`	Orders	Pickup/delivery, menu info	Vendor Order, Event Setup
`hotel` / `motel`	Bookings	Rooms, guest services	Universal only
`medical`	Cases	Appointments, intake, Rx	Insurance, Prescription
`dental`	Cases	Appointments, intake	Insurance Verification
`salon` / `spa`	Appointments	Appointments, pricing	Universal only
`legal`	Consultations	Consults, case intake	Universal only
`hvac` / `plumbing`	Service Orders	Service, emergency, quotes	Parts Order, Vehicle
`auto_service`	Service Orders	Scheduling, quotes	Parts Order, Vehicle

Industry-Specific Pipeline Stages

Each industry has its own set of pipeline stages for the deals/CRM system:

Medical / Dental: Inquiry -> Scheduled -> Confirmed -> Completed | No-Show | Cancelled

Restaurant: Inquiry -> Reserved -> Confirmed -> Seated -> Completed | No-Show | Cancelled

Hotel / Motel: Inquiry -> Reserved -> Checked In -> Checked Out | No-Show | Cancelled

Salon: Inquiry -> Booked -> Confirmed -> Completed | No-Show | Cancelled

Auto Service: Inquiry -> Quoted -> Scheduled -> In Progress -> Completed | Cancelled

Default (all others): New -> Contacted -> Qualified -> Proposal -> Negotiation -> Won | Lost

---

2. Architecture

Component Overview

Lumentra consists of four main components plus supporting infrastructure:

```
                    +------------------+
                    |   SignalWire SIP  |
                    |   (Telephony)    |
                    +--------+---------+
                             |
                    +--------v---------+
                    |   LiveKit SIP    |
                    |   (SIP Bridge)   |
                    +--------+---------+
                             |
                    +--------v---------+
                    |  LiveKit Server  |
                    |   (WebRTC SFU)   |
                    +--------+---------+
                             |
+-------------+     +--------v---------+     +----------------+
|  Dashboard  | <-->|   Lumentra API   | <-->|   PostgreSQL   |
| (Next.js)   |     |   (Hono/Node)    |     |   (Supabase)   |
| Port 3000   |     |   Port 3100      |     +----------------+
+-------------+     +--------+---------+
                             |
                    +--------v---------+
                    | LiveKit Agent    |
                    | (Python v1.4)    |
                    +------------------+
```

Service Stack

Component	Technology	Port	Purpose
lumentra-api	Node.js / Hono / TS	3100	REST API, business logic
lumentra-dashboard	Next.js 16	3000	Admin UI, analytics
lumentra-agent	Python / LiveKit 1.4	–	Voice AI agent
LiveKit Server	livekit-server v1.8	7880-7881	WebRTC SFU
LiveKit SIP	livekit/sip v1.8	5060	SIP-to-WebRTC bridge
Redis	redis:7-alpine	6379	LiveKit coordination
PostgreSQL	Supabase PG 16	5432	Primary database

Voice Pipeline

The voice pipeline for each call follows this flow:

1. **Inbound call** arrives at SignalWire phone number
2. **SignalWire** sends the call to the API's `/sip/forward` endpoint
3. **API** returns TwiML directing the call to LiveKit SIP
4. **LiveKit SIP** bridges the SIP call into a LiveKit room
5. **LiveKit Agent** joins the room and handles the conversation:
   - **STT:** Deepgram nova-3 (multi-language, smart format)
   - **LLM:** OpenAI gpt-4.1-mini (temperature 0.8)
   - **TTS:** Cartesia Sonic-3 (speed 0.95)
   - **Turn Detection:** LiveKit multilingual model + Silero VAD
6. **Agent** calls tools via the Internal API (`/internal/voice-tools/:action`)
7. **Agent** logs the call via `/internal/calls/log` when complete
8. **Post-call automation** runs asynchronously to create deals, tasks, and update contacts

API Route Structure

All API routes are served from `lumentra-api/src/index.ts`:

Route Prefix	Auth Type	Description
`/health`	None	Health checks and ping
`/api/chat`	None	Public chat widget
`/sip/forward`	None	SignalWire SIP webhook
`/internal/*`	Bearer (INTERNAL_API_KEY)	Agent communication
`/api/setup/*`	User JWT (no tenant)	Setup wizard
`/api/tenants`	User JWT (no tenant)	Tenant listing/creation
`/api/*`	User JWT + X-Tenant-ID	All protected routes

---

3. Dashboard Walkthrough

The Lumentra dashboard is a Next.js 16 application that provides the administrative interface for managing tenants, viewing call analytics, handling escalations, and configuring the voice agent. It uses a provider hierarchy of ThemeProvider -> AuthProvider -> TenantProvider -> ConfigProvider -> IndustryProvider -> EscalationProvider.

Dashboard Layout

The dashboard uses a responsive layout with the following structure:

- **Sidebar (desktop):** Collapsible navigation rail on the left side with links to all major sections. Visible on screens wider than 768px.
- **TopBar:** Persistent header bar with tenant selector, user menu, and contextual actions.
- **Main Content:** The central area where page content renders, with scroll overflow.
- **Escalation System:** A floating dock and slide-out panel on the right side for managing escalation queue items in real-time.
- **Mobile Navigation:** A bottom tab bar for mobile devices, replacing the sidebar.
- **Skip Links:** Accessibility skip-links for keyboard navigation ("Skip to main content", "Skip to navigation").

The layout redirects unauthenticated users to `/login` and users with no tenants to `/setup`.

3.1 Dashboard Home (`/dashboard`)

The main dashboard page is a three-panel layout:

Left Panel – System Health (collapsible, 256px wide):

- Real-time system health metrics
- Database connection status
- Cache hit rates and tenant count
- API response latency
- Collapse to a narrow 48px strip showing a green dot and "Metrics" label

Center Panel – Waveform:

- Live call visualization
- Active call status and duration
- Animated audio waveform during active calls

Right Panel – Activity Log (collapsible, 320px wide):

- Real-time feed of system events

- Call completions, bookings, escalations
- Timestamp and event type for each entry
- Collapse to a narrow strip with a pulsing blue dot and “Activity” label

Setup Incomplete Banner:

When a tenant has not completed the setup wizard, a banner appears at the top prompting the user to finish configuration.

Mobile View:

On mobile devices, the three panels are replaced by a tab navigation at the bottom with “Live Call”, “Metrics”, and “Activity” tabs.

3.2 Calls Page (`/calls`)

The calls page provides a comprehensive view of all inbound and outbound calls:

- **Call List:** Sortable, filterable table of all calls with columns for caller phone, caller name, direction, status, duration, outcome type, and timestamp.
- **Filters:** Filter by status (completed, missed, failed), outcome type (booking, inquiry, support, escalation, hangup), date range, and search by phone or name.
- **Call Detail:** Click any call to view full details including transcript, summary, sentiment score, detected intents, linked contact, and linked booking.
- **Statistics Bar:** Top-of-page stats showing total calls, average duration, success rate, and booking conversion rate.

3.3 Contacts Page (`/contacts`)

Full CRM contact management:

- **Contact List:** Searchable, sortable list of all contacts with name, phone, email, status, lead status, engagement score, and last contact date.
- **Contact Detail Panel:** Slide-out panel showing full contact profile, activity history, notes, linked calls, and linked bookings.
- **Actions:** Create, edit, merge duplicates, import (up to 10,000 records), export (JSON or CSV).
- **Status Management:** Toggle between active, inactive, do-not-contact, and VIP statuses.
- **Tags:** Add and remove tags for segmentation and filtering.
- **Notes:** Add notes with types (general, preference, complaint, compliment, follow_up, internal) and pin important notes.
- **Engagement Score:** Visual indicator (0-100) showing contact engagement level based on recency, frequency, completion rate, and value.

3.4 Calendar Page (`/calendar`)

Booking and appointment management with calendar visualization:

- **Calendar View:** Month, week, and day views showing all bookings color-coded by status.
- **Day Summary:** Quick stats for selected day showing total bookings, confirmed, pending, and completed counts.
- **Booking Detail:** Click any booking to view or edit details including customer name, phone, service type, date, time, status, confirmation code, and notes.
- **Status Transitions:** Confirm, complete, mark as no-show, cancel, or reschedule bookings directly from the calendar.
- **Upcoming List:** Sidebar showing next upcoming bookings sorted by date and time.

3.5 Deals Page (`/deals`)

Sales pipeline management with Kanban board:

- **Kanban Board:** Visual pipeline with columns for each stage (industry-specific). Drag and drop deals between stages.
- **Deal Cards:** Each card shows deal name, company, amount, contact name, source, and age.
- **Filters:** Filter by stage, source (call, web, manual, import), contact, date range, and search.
- **Deal Detail:** Click any deal to view/edit full details including linked contact, linked call, description, expected close date, and amount.
- **Auto-Created Deals:** Deals created by post-call automation are marked with source "call" and created_by "auto".

3.6 Tasks Page (`/tasks`)

Task management for follow-ups and action items:

- **Task List:** Filterable list showing title, type, priority, due date, status, assigned contact, and source.
- **Task Types:** Industry-specific types (e.g., follow_up, call_back, email, meeting, review, custom, plus industry-specific types like insurance_verification, prescription_refill, vendor_order).
- **Priority Levels:** urgent, high, medium, low – with visual color coding.
- **Quick Actions:** Mark complete, delete, edit due date and priority inline.
- **Counts Bar:** Top stats showing total, overdue, due today, and upcoming counts.
- **Auto-Created Tasks:** Tasks created by post-call automation for escalated calls and missed calls.

3.7 Escalations Page (`/escalations`)

Real-time escalation queue management:

- **Stats Cards:** Four cards showing Waiting count, High Priority count (urgent + high), Average Wait Time, and Resolved Today count.
- **Filter Bar:** Filter by status (all, waiting, in-progress, callback-scheduled, resolved) and priority (all, urgent, high, normal, low).
- **Escalation Cards:** Each card shows caller info, reason for escalation, priority badge, wait time, and action buttons.
- **Actions:** "Take Call" button for waiting escalations, "Schedule Callback" to open the callback scheduling panel, and click to open detailed panel.
- **Summary Footer:** Shows count of displayed vs total escalations and number of scheduled callbacks.

3.8 Pending Bookings Page (`/pending`)

Review and confirm booking requests collected by the voice agent when in assisted mode:

- **Stats Cards:** Pending count, Confirmed count, Rejected count, Today count.
- **Tabs:** "Pending" tab (shows only pending items) and "All" tab (shows all statuses).
- **Booking Cards:** Each card shows caller name, requested service, date/time, and action buttons.
- **Actions:** Confirm or reject each booking. View the call transcript that generated the booking.
- **Industry Labels:** Uses industry-specific terminology (e.g., "Reservations" for restaurants, "Appointments" for medical).

3.9 Resources Page (`/resources`)

Staff, rooms, and equipment management:

- **Resource List:** Filterable by type (staff, room, equipment), active/bookable status.
- **Resource Detail:** Name, type, description, active status, bookable flag, display order.
- **CRUD Operations:** Create, edit, reorder, and deactivate resources.
- **Availability Check:** View resource availability for a given date.

3.10 Notifications Page (`/notifications`)

Notification management and template configuration:

- **Notification List:** All sent and queued notifications with status, channel (SMS/email), recipient, and timestamp.
- **Template Management:** View and edit notification templates with variable previews.
- **Preferences:** Configure per-tenant notification preferences (which events trigger notifications).
- **Send/Preview:** Send test notifications or preview template rendering before sending.

3.11 Analytics Page (`/analytics`)

Detailed call and business analytics:

- **Time Series Charts:** Call volume over time (daily, weekly, monthly).
- **Peak Hours Analysis:** Heat map or bar chart showing busiest call hours.
- **Outcome Breakdown:** Pie/bar chart of call outcomes (booking, inquiry, support, escalation, hangup).
- **Duration Distribution:** Average call duration trends.
- **Conversion Metrics:** Booking conversion rate, escalation rate.

3.12 Workstation Page (`/workstation`)

Unified workspace combining multiple views for operators who handle calls, escalations, and bookings simultaneously. Provides a consolidated view of active escalations, upcoming bookings, and recent calls in a single screen.

3.13 Settings Pages (`/settings`)

The settings hub provides links to all configuration sub-pages:

3.13.1 Business Settings (`/settings/business`) Core business information: business name, industry, location address, location city.

3.13.2 Assistant Settings (`/settings/assistant`) Voice agent identity configuration:

- **Agent Name:** Text input with name suggestions based on personality (e.g., “Sarah”, “James” for professional; “Emma”, “Alex” for friendly; “Kate”, “Sam” for efficient).
- **Voice Selection:** Six voice options in a 3D card grid:
  - Sarah (Female, Professional)
  - Emma (Female, Friendly)
  - Maya (Female, Warm)
  - James (Male, Professional)
  - Alex (Male, Friendly)
  - David (Male, Calm)
  - Each voice has a play/pause preview button
- **Personality Selection:** Three wobble-card options with live greeting preview:
  - **Professional:** Formal and business-like. Example: “Good afternoon, thank you for calling {business}. This is {name}, how may I assist you today?”
  - **Friendly:** Warm and conversational. Example: “Hey there! Thanks for calling {business}. I’m {name} - what can I help you with?”
  - **Efficient:** Direct and to the point. Example: “{business}, this is {name}. How can I help?”

3.13.3 Operating Hours (`/settings/hours`) Business hours configuration:

- **Timezone Selection:** US timezones (Eastern, Central, Mountain, Pacific, Alaska, Hawaii).
- **Same Hours Toggle:** When enabled, applies Monday’s schedule to all weekdays.

- **Per-Day Schedule:** Each day can be set to Open (with open/close times in 30-minute increments), Closed, or 24 Hours.
- **Default Schedule:** Monday-Friday 9:00 AM - 5:00 PM, Saturday-Sunday closed.
- **After-Hours Behavior:** Four options:
  - Still answer and let callers know we're closed (takes messages)
  - Take messages only (skip conversation, collect contact info)
  - Send to voicemail (play message and record)
  - Forward to emergency contact (connect urgent calls to escalation contact)

3.13.4 Phone Settings (`/settings/phone`) Phone number configuration and status:

- **Current Number Display:** Shows the active phone number with status badge (Active, Pending, Porting, Failed).
- **Setup Type:** Shows whether the number is New (provisioned), Ported (transferred), or Forwarded.
- **Port Status Tracking:** For ported numbers, shows status (Draft, Submitted, Pending Review, Approved, Rejected, Completed) with estimated completion date and rejection reasons.
- **Forwarding Instructions:** For forwarded numbers, shows carrier-specific instructions (AT&T, Verizon, T-Mobile, Sprint) with the forwarding destination number and copy button.

3.13.5 Integrations (`/settings/integrations`) External system connections:

- **Integration Mode Display:** Shows current mode (External System, Built-in Scheduling, or Assisted Mode).
- **Available Integrations:** Grouped by type:
  - **Calendars:** Google Calendar, Microsoft Outlook
  - **Booking Systems:** Calendly, Acuity Scheduling, Square Appointments, Vagaro (salon), Mindbody (salon)
  - **Point of Sale:** Toast (restaurant), OpenTable (restaurant)
- **OAuth Connect Flow:** Click "Connect" to open OAuth popup window (600x700px). On success, the integration status updates to "Connected". On error, an error message is displayed.
- **Disconnect:** Remove an active integration with a single click.
- **Status Indicators:** Connected (green), Expired (amber), Disconnected (gray), Error (red).

3.13.6 Capabilities (`/settings/capabilities`) Industry-specific feature toggles. Available capabilities vary by industry:

- Restaurant: reservations, takeaway orders, menu info, specials
- Medical: appointments, patient intake, insurance verification, prescription refills
- Dental: appointments, new patient intake, insurance
- Salon/Spa: appointments, services pricing
- Hotel: room reservations, guest services, amenities
- HVAC/Plumbing: service appointments, emergency dispatch, quotes
- Legal: consultations, case intake

3.13.7 Escalation Settings (`/settings/escalation`) Configure when and how calls are escalated to humans. See Section 18 for full details.

3.13.8 Promotions (`/settings/promotions`) Manage active promotions that the voice agent can mention to callers. CRUD interface with active/inactive toggle.

3.14 Profile Page (`/profile`)

User account management with four tabs:

- **Profile Tab:** Display name, email, phone number, role (read-only), avatar upload.

- **Security Tab:** Change password (requires current password, minimum 8 characters), two-factor authentication toggle (not yet enabled), active sessions list, sign out all devices, danger zone (delete account).
- **Notifications Tab:** Email preferences (call summaries, escalation alerts, weekly reports, billing, product updates) and push preferences (escalations, missed calls, system alerts) with sound volume control.
- **Preferences Tab:** Theme selector (light/dark/system), language selector, accessibility options (reduce motion, high contrast), keyboard shortcuts display (Cmd+K: command palette, Cmd+B: toggle sidebar, Cmd+/: show shortcuts, Esc: close dialog).

3.15 Setup Wizard (`/setup`)

Multi-step onboarding flow for new tenants:

1. **Business:** Business name, industry selection
2. **Capabilities:** Industry-specific feature toggles
3. **Details:** Location address, city, custom instructions
4. **Integrations:** Calendar/booking system connection
5. **Assistant:** Agent name, voice, personality
6. **Phone:** Phone number setup (new, port, forward, SIP)
7. **Hours:** Operating hours and timezone
8. **Escalation:** Escalation contacts and triggers
9. **Review:** Summary of all settings with edit links

The wizard saves progress at each step. Users can navigate back to previous steps. The setup is marked complete when the review step is submitted.

---

4. Tenant Management

Understanding Tenants

A tenant represents a single business on the platform. Each tenant has:

- A unique UUID identifier
- A unique phone number (maps incoming calls to the correct business)
- Voice agent configuration (name, personality, greetings)
- Operating hours and timezone
- Escalation settings
- Feature flags and subscription tier
- CRM data (contacts, calls, bookings, deals, tasks)
- Industry-specific pipeline configuration

Tenant Lifecycle

1. **Creation:** User signs up and creates a tenant via the setup wizard or API
2. **Setup:** Tenant progresses through setup wizard steps (business -> capabilities -> details -> integrations -> assistant -> phone -> hours -> escalation -> review)
3. **Active:** Setup complete, tenant is receiving and handling calls
4. **Suspended:** Tenant disabled by admin (calls still arrive but are not answered)

Creating a Tenant via API

Endpoint: `POST /api/tenants`

Required Headers:

```
Authorization: Bearer <supabase_jwt>
```

Required Fields: `business_name`, `phone_number`, `industry`

Request Body:

```
{
  "business_name": "Bella Italia Restaurant",
  "phone_number": "+15551234567",
  "industry": "restaurant",
  "agent_name": "Sofia",
  "timezone": "America/New_York",
  "agent_personality": {
    "tone": "friendly"
  },
  "voice_config": {
    "voiceId": "female_friendly"
  }
}
```

Response (201):

```
{
  "id": "uuid-here",
  "business_name": "Bella Italia Restaurant",
  "phone_number": "+15551234567",
  "industry": "restaurant",
  "agent_name": "Sofia",
  "is_active": true,
  "status": "draft"
}
```

Updating a Tenant

Endpoint: `PUT /api/tenants/:id`

Required Role: `owner` or `admin`

Updatable Fields: All tenant columns except `id`, `created_at`. The `agent_personality` field must be a JSONB object (not a string).

Listing Tenants

Endpoint: `GET /api/tenants`

Returns all tenants the authenticated user has membership in. Does not require X-Tenant-ID header.

Phone Number Updates

Endpoint: `PATCH /api/tenants/:id/phone`

Required Role: `owner` or `admin`

Updates the tenant's phone number and invalidates the tenant cache so that incoming calls route correctly. The phone number must be unique across all tenants.

---

5. User and Role Management

Authentication

Lumentra uses Supabase Auth for user authentication. Users sign up with email/password and receive a JWT token. The JWT is passed as a Bearer token in the Authorization header.

Three authentication middleware levels are available:

1. **authMiddleware:** Requires valid JWT AND X-Tenant-ID header. Verifies the user has membership in the specified tenant. Used for all tenant-scoped API routes.
2. **userAuthMiddleware:** Requires valid JWT only. Does not require tenant context. Used for setup wizard and tenant listing.
3. **optionalAuthMiddleware:** JWT is optional. If present, the user is attached to the request. Used for public endpoints that provide enhanced functionality for logged-in users.

Roles

The `tenant_members` table controls access with four roles:

Role	Description	Typical Use
`owner`	Full access, manage members, delete	Business owner
`admin`	Full access, no tenant deletion	Office manager
`member`	View data, handle escalations	Staff member
`readonly`	Read-only dashboard and reports	Reporting user

Role Enforcement

Role checks are applied at the route level using the `requireRole()` middleware:

```
requireRole("owner", "admin")  // Only owner or admin can access
```

Routes that require specific roles: - **Tenant updates:** `owner` or `admin` - **Member management:** `owner` or `admin` - **Phone number changes:** `owner` or `admin` - **Settings changes:** `owner` or `admin` - **Data viewing:** all roles - **Escalation handling:** `owner`, `admin`, or `member`

Member Management

Add a member: `POST /api/tenants/:id/members`

```
{
  "user_id": "supabase-user-uuid",
  "role": "member"
}
```

Update a member's role: `PATCH /api/tenants/:id/members/:userId`

```
{
  "role": "admin"
}
```

Remove a member: `DELETE /api/tenants/:id/members/:userId`

List members: `GET /api/tenants/:id/members`

---

6. System Configuration

Voice Pipeline Configuration

Each tenant has a `voice_pipeline` column that determines call routing:

Value	Description
`custom`	Legacy SignalWire WebSocket pipeline
`livekit`	LiveKit Agents SIP pipeline (default)

The LiveKit pipeline is the active default. The custom pipeline is retained for backward compatibility.

Voice Agent Settings

The voice agent is configured through three tenant columns:

`agent_personality` (JSONB):

```
{
  "tone": "professional"  // "professional", "friendly", or "efficient"
}
```

`voice_config` (JSONB):

```
{
  "voiceId": "female_professional"  // ID of the selected voice
}
```

`greeting_standard` / `greeting_after_hours` / `greeting_returning` (TEXT):

Custom greeting messages. If not set, the system prompt builder generates appropriate greetings based on the personality and business name.

Operating Hours

`operating_hours` (JSONB):

```
{
  "timezone": "America/New_York",
  "schedule": [
    { "day": 0, "enabled": true, "openTime": "09:00", "closeTime": "17:00" },
    { "day": 1, "enabled": true, "openTime": "09:00", "closeTime": "17:00" },
    { "day": 2, "enabled": true, "openTime": "09:00", "closeTime": "17:00" },
    { "day": 3, "enabled": true, "openTime": "09:00", "closeTime": "17:00" },
    { "day": 4, "enabled": true, "openTime": "09:00", "closeTime": "17:00" },
    { "day": 5, "enabled": false, "openTime": "09:00", "closeTime": "17:00" },
    { "day": 6, "enabled": false, "openTime": "09:00", "closeTime": "17:00" }
  ],
  "holidays": []
}
```

Day indices: 0=Monday through 6=Sunday. Times are in 24-hour format.

Feature Flags

`features` (JSONB):

```
{
  "sms_enabled": true,
  "email_enabled": true,
  "recording_enabled": true,
  "voicemail_enabled": true,
  "chat_widget_enabled": true
}
```

Max Call Duration

`max_call_duration_seconds` (INTEGER):

Default is 900 (15 minutes). The LiveKit agent enforces this limit and gracefully ends the call when reached.

---

7. API Endpoint Reference

This section documents every API endpoint in the system, organized by route file. Each entry includes the HTTP method, path, authentication requirements, request/response formats, and behavior details.

7.1 Health Routes (`/health`)

No authentication required.

`GET /health` Returns system health status including database connectivity and cache statistics.

Response (200):

```
{
  "status": "healthy",
  "database": "connected",
  "cache": {
    "tenants": 5,
    "lastRefresh": "2026-03-02T10:00:00Z"
  },
  "uptime": 86400
}
```

`GET /health/ping` Simple liveness check.

Response (200):

```
{ "pong": true }
```

7.2 Chat Routes (`/api/chat`)

No authentication required (public chat widget).

`POST /api/chat/message` Send a message to the chat widget. Uses a multi-provider LLM fallback chain: Gemini (primary) -> OpenAI -> Groq.

Request Body:

```
{
  "message": "What are your hours?",
  "session_id": "abc123",
  "tenant_id": "uuid"
}
```

Response (200):

```
{
  "reply": "We're open Monday through Friday, 9 AM to 5 PM.",
  "provider": "gemini"
}
```

`GET /api/chat/config` Returns chat widget configuration for embedding.

`GET /api/chat/health` Returns health status with per-provider availability.

7.3 Internal Routes (`/internal`)

Authentication: Bearer token (INTERNAL_API_KEY). Used exclusively by the LiveKit Python agent.

`GET /internal/tenants/by-phone/:phone` Looks up tenant configuration by phone number. Returns the full tenant config plus a pre-built system prompt for the voice agent.

Response (200):

```
{
  "id": "tenant-uuid",
  "business_name": "Bella Italia",
  "industry": "restaurant",
  "agent_name": "Sofia",
  "phone_number": "+15551234567",
  "voice_config": { "voiceId": "female_friendly" },
  "agent_personality": { "tone": "friendly" },
  "greeting_standard": "...",
  "greeting_after_hours": "...",
  "greeting_returning": "...",
  "timezone": "America/New_York",
  "operating_hours": { "..." },
  "escalation_enabled": true,
  "escalation_phone": "+15559876543",
  "escalation_triggers": ["speak to manager", "emergency"],
  "features": { "..." },
  "voice_pipeline": "livekit",
  "max_call_duration_seconds": 900,
  "system_prompt": "You are Sofia, a friendly AI assistant for Bella Italia..."
}
```

`POST /internal/voice-tools/:action` Routes tool calls from the Python agent to the Node.js tool execution functions.

Available actions:

- `check_availability` – Check available time slots for a given date
- `create_booking` – Create a confirmed booking with confirmation code
- `create_order` – Create a pickup/delivery order (restaurant/pizza)

- `transfer_to_human` – Initiate transfer to escalation phone
- `end_call` – Signal call completion
- `log_note` – Save a note to the caller's contact record

Request Body:

```
{
  "tenant_id": "uuid",
  "call_sid": "call-identifier",
  "caller_phone": "+15551234567",
  "escalation_phone": "+15559876543",
  "args": {
    "date": "2026-03-05",
    "time": "14:00",
    "customer_name": "John Smith",
    "customer_phone": "+15551234567"
  }
}
```

Response (200):

```
{
  "result": {
    "success": true,
    "booking_id": "uuid",
    "confirmation_code": "ABC123",
    "message": "I've booked your appointment for Wednesday, March 5th at 2 PM."
  }
}
```

`POST /internal/calls/log` Saves a completed call record. Automatically finds or creates a contact by caller phone. Triggers post-call automation asynchronously.

Request Body:

```
{
  "tenant_id": "uuid",
  "call_sid": "call-identifier",
  "caller_phone": "+15551234567",
  "caller_name": "John Smith",
  "direction": "inbound",
  "status": "completed",
  "started_at": "2026-03-02T10:00:00Z",
  "ended_at": "2026-03-02T10:05:30Z",
  "duration_seconds": 330,
  "outcome_type": "booking",
  "outcome_success": true,
  "transcript": "Full conversation transcript...",
  "summary": "Caller booked an appointment for March 5th at 2 PM.",
  "sentiment_score": 0.85,
  "intents_detected": ["booking", "availability_check"],
  "recording_url": "https://...",
  "cost_cents": 2
}
```

Status Mapping: The agent status is mapped to database constraints: - `completed` -> `completed` - `failed` -> `failed` - `transferred` -> `completed` (with escalation outcome) - `no-answer` -> `missed` - default -> `completed`

7.4 Tenant Routes (`/api/tenants`)

Authentication: User JWT. X-Tenant-ID required for individual tenant operations.

`GET /api/tenants` List all tenants the authenticated user belongs to. No X-Tenant-ID required.

`POST /api/tenants` Create a new tenant. The creating user becomes the `owner`. No X-Tenant-ID required.

`GET /api/tenants/:id` Get a single tenant's full configuration.

`PUT /api/tenants/:id` Update a tenant. **Requires role: owner or admin.**

`PATCH /api/tenants/:id/phone` Update tenant phone number and invalidate cache. **Requires role: owner or admin.**

`GET /api/tenants/:id/members` List all members of a tenant.

`POST /api/tenants/:id/members` Add a member to a tenant. **Requires role: owner or admin.**

`PATCH /api/tenants/:id/members/:userId` Update a member's role. **Requires role: owner or admin.**

`DELETE /api/tenants/:id/members/:userId` Remove a member from a tenant. **Requires role: owner or admin.**

7.5 Call Routes (`/api/calls`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/calls` List calls with filters and pagination.

Query Parameters:

Parameter	Type	Description
`search`	string	Search caller phone or name
`status`	string	Filter by status (comma-separated)
`outcome_type`	string	Filter by outcome type
`direction`	string	"inbound" or "outbound"
`start_date`	string	ISO date, calls after this date
`end_date`	string	ISO date, calls before this date
`limit`	number	Page size (default 50)
`offset`	number	Pagination offset
`sort_by`	string	Sort column
`sort_order`	string	"asc" or "desc"

`GET /api/calls/stats` Aggregate call statistics for the tenant.

`GET /api/calls/analytics` Time series and distribution analytics.

Query Parameters:

- `period`: "day", "week", or "month"
- `start_date`, `end_date`: date range

Returns time series data, peak hours analysis, and outcome distribution.

`GET /api/calls/recent` Returns the most recent calls (shortcut for common dashboard use).

`GET /api/calls/:id` Get a single call with full details including transcript, linked contact, and linked booking.

7.6 Contact Routes (`/api/contacts`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/contacts` List contacts with search, filters, and pagination.

Query Parameters:

Parameter	Type	Description
`search`	string	Search name, phone, or email
`status`	string	Filter: active, inactive, dnc, vip
`lead_status`	string	Filter by lead status
`tags`	string	Comma-separated tag filter
`limit`	number	Page size
`offset`	number	Pagination offset

`POST /api/contacts` Create a new contact.

Required Fields: At least one of phone or `email`.

`GET /api/contacts/:id` Get a single contact with full profile.

`PUT /api/contacts/:id` Update a contact's information.

`DELETE /api/contacts/:id` Soft-delete a contact.

`GET /api/contacts/lookup/phone/:phone` Look up a contact by phone number.

`GET /api/contacts/lookup/email/:email` Look up a contact by email address.

`POST /api/contacts/find-or-create` Find an existing contact by phone or create a new one. Used internally during call logging.

`PATCH /api/contacts/:id/status` Update a contact's status (active, inactive, dnc, vip).

`PATCH /api/contacts/:id/tags` Update a contact's tags array.

`GET /api/contacts/:id/notes` List all notes for a contact.

POST /api/contacts/:id/notes Add a note to a contact. Note types: general, preference, complaint, compliment, follow_up, internal.

GET /api/contacts/:id/history Get activity history for a contact (calls, bookings, notes, status changes).

GET /api/contacts/:id/bookings List all bookings for a contact.

GET /api/contacts/:id/calls List all calls from a contact.

POST /api/contacts/import Bulk import contacts (up to 10,000 records per request).

Request Body:

```
{
  "contacts": [
    { "name": "John Smith", "phone": "+15551234567", "email": "john@example.com", "tags": ["v
  ]
}
```

GET /api/contacts/export Export all contacts. Supports `format` query parameter: `json` (default) or `csv`.

POST /api/contacts/merge Merge two duplicate contacts into one.

```
{
  "primary_id": "uuid-to-keep",
  "secondary_id": "uuid-to-merge"
}
```

GET /api/contacts/:id/engagement Get the engagement score breakdown for a contact.

7.7 Booking Routes (/api/bookings)

Authentication: User JWT + X-Tenant-ID.

GET /api/bookings List bookings with filters and pagination.

POST /api/bookings Create a new booking manually.

GET /api/bookings/:id Get a single booking.

PUT /api/bookings/:id Update a booking.

GET /api/bookings/upcoming Get upcoming bookings sorted by date.

GET /api/bookings/calendar Get bookings for calendar view. Query parameters: `start_date`, `end_date`.

GET /api/bookings/day-summary Get booking statistics for a specific day.

PATCH /api/bookings/:id/confirm Confirm a pending booking.

PATCH /api/bookings/:id/complete Mark a booking as completed.

PATCH /api/bookings/:id/no-show Mark a booking as no-show.

PATCH /api/bookings/:id/cancel Cancel a booking.

PATCH /api/bookings/:id/reschedule Reschedule a booking to a new date/time.

```
{
  "booking_date": "2026-03-10",
  "booking_time": "15:00"
}
```

7.8 Availability Routes (/api/availability)

Authentication: User JWT + X-Tenant-ID.

GET /api/availability/slots List availability slots with optional date range filter.

POST /api/availability/slots Create availability slots.

PUT /api/availability/slots/:id Update an availability slot.

DELETE /api/availability/slots/:id Delete an availability slot.

GET /api/availability/check Check availability for a specific date. Returns available time slots.

POST /api/availability/generate Generate availability slots from operating hours for the next 30 days.

PATCH /api/availability/slots/:id/block Block a specific time slot (mark unavailable).

PATCH /api/availability/slots/:id/unblock Unblock a previously blocked time slot.

7.9 Resource Routes (/api/resources)

Authentication: User JWT + X-Tenant-ID.

GET /api/resources List resources with optional filters.

Query Parameters:

- `type`: Filter by type (staff, room, equipment)
- `active`: Filter by active status (true/false)
- `bookable`: Filter by bookable status (true/false)

POST /api/resources Create a new resource.

GET /api/resources/:id Get a single resource.

`PUT /api/resources/:id` Update a resource.

`DELETE /api/resources/:id` Delete a resource.

`GET /api/resources/:id/availability` Check a resource's availability for a given date.

`PATCH /api/resources/reorder` Reorder resources (update display_order).

7.10 Escalation Routes (`/api/escalation`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/escalation/queue` Get the current escalation queue.

`POST /api/escalation/queue/:id/take` Take ownership of an escalation (mark as in-progress).

`POST /api/escalation/queue/:id/resolve` Resolve an escalation.

`POST /api/escalation/queue/:id/schedule-callback` Schedule a callback for an escalation.

`GET /api/escalation/contacts` List escalation contacts.

`POST /api/escalation/contacts` Add an escalation contact.

`PUT /api/escalation/contacts/:id` Update an escalation contact.

`DELETE /api/escalation/contacts/:id` Remove an escalation contact.

`GET /api/escalation/triggers` Get escalation trigger configuration.

`PUT /api/escalation/triggers` Update escalation triggers.

`PATCH /api/escalation/contacts/reorder` Reorder escalation contacts.

7.11 Notification Routes (`/api/notifications`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/notifications` List notifications with status filter.

`GET /api/notifications/:id` Get a single notification.

`POST /api/notifications/send` Send a notification immediately.

```
{
  "template_id": "booking_confirmation",
  "channel": "sms",
  "recipient_phone": "+15551234567",
  "variables": {
    "customer_name": "John",
    "booking_date": "March 5",
    "booking_time": "2:00 PM",
    "confirmation_code": "ABC123"
  }
}
```

POST /api/notifications/preview Preview a rendered notification template without sending.

GET /api/notifications/templates List all notification templates.

POST /api/notifications/templates Create a new notification template.

PUT /api/notifications/templates/:id Update a notification template.

DELETE /api/notifications/templates/:id Delete a notification template.

GET /api/notifications/preferences Get notification preferences for the tenant.

PUT /api/notifications/preferences Update notification preferences.

POST /api/notifications/process-queue Manually trigger processing of the notification queue.

7.12 Dashboard Routes (/api/dashboard)

Authentication: User JWT + X-Tenant-ID.

GET /api/dashboard/metrics System health metrics: API latency, call/booking counts, active sessions.

GET /api/dashboard/activity Recent activity log entries.

GET /api/dashboard/stats Aggregate statistics for the dashboard home page.

GET /api/dashboard/sessions Active session information.

7.13 Deal Routes (/api/deals)

Authentication: User JWT + X-Tenant-ID.

GET /api/deals List deals with search, filters, and pagination.

Query Parameters:

Parameter	Type	Description
`search`	string	Search deal name or company
`stage`	string	Filter by stage (comma-separated)
`contact_id`	string	Filter by linked contact
`source`	string	Filter: call, web, manual, import
`start_date`	string	Created after this date
`end_date`	string	Created before this date
`limit`	number	Page size
`offset`	number	Pagination offset
`sort_by`	string	Sort column
`sort_order`	string	"asc" or "desc"

`GET /api/deals/pipeline` Get pipeline view grouped by stage for Kanban board display. Returns stages array with deals grouped by stage, using industry-specific stage configuration.

Response (200):

```
{
  "stages": [
    {
      "id": "inquiry",
      "label": "Inquiry",
      "color": "blue",
      "deals": [...]
    },
    {
      "id": "reserved",
      "label": "Reserved",
      "color": "cyan",
      "deals": [...]
    }
  ]
}
```

`GET /api/deals/:id` Get a single deal.

`POST /api/deals` Create a new deal.

Request Body:

```
{
  "name": "John Smith - New Reservation",
  "stage": "inquiry",
  "source": "call",
  "contact_id": "uuid",
  "call_id": "uuid",
  "amount_cents": 5000,
  "expected_close": "2026-03-15"
}
```

Validation:

- `name` is required (min 1 character)
- `stage` is optional (defaults to industry default stage)
- `amount_cents` must be a non-negative integer
- `contact_id` and `call_id` must be valid UUIDs if provided

- `source` must be one of: call, web, manual, import

`PUT /api/deals/:id` Update a deal.

`PATCH /api/deals/:id/stage` Update deal stage only (optimized for Kanban drag-and-drop). Validates that the target stage exists in the tenant's industry pipeline.

```
{
  "stage": "reserved",
  "sort_index": 2
}
```

Error (400) if invalid stage:

```
{
  "error": "Invalid stage \"invalid\" for industry \"restaurant\". Valid stages: inquiry, res
}
```

`DELETE /api/deals/:id` Soft-delete (archive) a deal.

7.14 Task Routes (`/api/tasks`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/tasks` List tasks with search, filters, and pagination.

Query Parameters:

- `search`: Search task title
- `type`: Filter by type (comma-separated)
- `priority`: Filter by priority
- `status`: Filter by status (pending, completed)
- `contact_id`: Filter by linked contact
- `due_before`: Tasks due before this date
- `due_after`: Tasks due after this date

`GET /api/tasks/counts` Get task count breakdown by status and priority.

`GET /api/tasks/upcoming` Get tasks due in the next 7 days.

`GET /api/tasks/overdue` Get overdue tasks (due date in the past, not completed).

`GET /api/tasks/:id` Get a single task.

`POST /api/tasks` Create a new task.

```
{
  "title": "Follow up with John Smith",
  "type": "call_back",
  "priority": "high",
  "due_date": "2026-03-03",
  "contact_id": "uuid",
  "call_id": "uuid",
  "notes": "Caller asked about insurance coverage"
}
```

`PUT /api/tasks/:id` Update a task.

`PATCH /api/tasks/:id/complete` Mark a task as completed.

`DELETE /api/tasks/:id` Delete a task.

7.15 Voicemail Routes (`/api/voicemails`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/voicemails` List voicemails with filters.

`PATCH /api/voicemails/:id` Update voicemail status (pending -> reviewed -> callback_scheduled -> resolved).

`POST /api/voicemails/record` Generate TwiML for voicemail recording (called by telephony provider).

`POST /api/voicemails/callback/complete` Webhook for recording completion.

`POST /api/voicemails/callback/status` Webhook for voicemail status updates.

`POST /api/voicemails/callback/transcribe` Webhook for voicemail transcription completion.

`GET /api/voicemails/stats` Voicemail statistics (total, pending, reviewed, resolved).

7.16 Promotion Routes (`/api/promotions`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/promotions` List all promotions.

`POST /api/promotions` Create a new promotion.

`GET /api/promotions/:id` Get a single promotion.

`PUT /api/promotions/:id` Update a promotion.

`DELETE /api/promotions/:id` Delete a promotion.

`PATCH /api/promotions/:id/toggle` Toggle a promotion’s active/inactive state.

7.17 Capability Routes (`/api/capabilities`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/capabilities/options` Get industry-specific capability options. Returns the available capabilities for the tenant's industry.

Supported Industries:

restaurant, pizza, medical, dental, home_services, hvac, plumbing, legal, salon, spa, hotel

`GET /api/capabilities` Get the tenant's currently enabled capabilities.

`PUT /api/capabilities` Update the tenant's enabled capabilities.

7.18 Training Data Routes (`/api/training-data`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/training-data` List conversation logs for training data curation.

`GET /api/training-data/:id` Get a single conversation log with full message history.

`PUT /api/training-data/:id` Update conversation log metadata (quality_score, reviewed, flagged, tags, notes).

`POST /api/training-data/bulk-review` Mark multiple conversation logs as reviewed.

`GET /api/training-data/export` Export training data in various formats.

Query Parameters:

- `format`: Output format – `jsonl` (default), `sharegpt`, `alpaca`, `openai`
- `min_quality`: Minimum quality score filter (0.0 to 1.0)
- `scenario_type`: Filter by scenario type

`GET /api/training-data/stats` Training data statistics (total logs, reviewed count, average quality, format distribution).

7.19 Pending Booking Routes (`/api/pending-bookings`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/pending-bookings` List pending bookings with optional status filter.

`GET /api/pending-bookings/stats` Pending booking statistics.

`GET /api/pending-bookings/:id` Get a single pending booking.

`PUT /api/pending-bookings/:id/confirm` Confirm a pending booking.

`PUT /api/pending-bookings/:id/reject` Reject a pending booking.

`POST /api/pending-bookings/:id/convert` Convert a pending booking to a confirmed booking in the main bookings table.

7.20 Integration Routes (`/api/integrations`)

Authentication: User JWT + X-Tenant-ID.

`GET /api/integrations` List all integrations for the tenant.

`GET /api/integrations/providers` List available integration providers.

`GET /api/integrations/:provider/authorize` Start OAuth authorization flow. Returns an authorization URL for the provider.

Supported providers with OAuth:

- `google_calendar` – Google Calendar sync
- `outlook` – Microsoft Outlook Calendar sync
- `calendly` – Calendly scheduling integration

Response (200):

```
{
  "authUrl": "https://accounts.google.com/o/oauth2/v2/auth?..."
}
```

`GET /api/integrations/:provider/callback` OAuth callback handler. Exchanges authorization code for tokens, encrypts and stores them, creates the integration record. Renders an HTML page that posts a message to the parent window.

`DELETE /api/integrations/:id` Disconnect an integration (revokes token and removes record).

`POST /api/integrations/:id/refresh` Manually refresh an integration's OAuth token.

7.21 Setup Wizard Routes (`/api/setup`)

Authentication: User JWT (no tenant context required for initial steps).

`GET /api/setup/progress` Get the current setup progress for the user's tenant.

`POST /api/setup/step/:step` Save data for a specific setup step. Steps: business, capabilities, details, integrations, assistant, phone, hours, escalation, review.

Each step saves its data to the appropriate tenant columns and advances the `setup_step` field.

`POST /api/setup/complete` Mark setup as complete. Sets `setup_completed_at` and changes tenant status to `active`.

`POST /api/setup/go-back` Navigate back to a previous step without losing saved data.

7.22 Phone Configuration Routes (`/api/phone`)

Authentication: User JWT + X-Tenant-ID.

GET /api/phone/search Search for available phone numbers to provision.

Query Parameters:

- `area_code`: Desired area code
- `country`: Country code (default "US")

GET /api/phone/config Get the current phone configuration for the tenant.

POST /api/phone/provision Provision a new phone number from SignalWire.

POST /api/phone/port Submit a port request to transfer an existing number.

GET /api/phone/port/status Check the status of a pending port request.

POST /api/phone/forward Set up call forwarding from an existing number.

POST /api/phone/forward/verify Verify that call forwarding is working correctly.

POST /api/phone/sip Create a SIP endpoint for businesses with VOIP/PBX systems.

GET /api/phone/sip/status Check SIP endpoint status.

---

8. Database Table Reference

This section documents every table in the Lumentra database with all columns, types, constraints, and relationships. Tables are created across 19 migration files.

8.1 tenants

Primary configuration table for each business.

Column	Type	Default	Description
`id`	UUID	`gen_random_uuid()`	Primary key
`business_name`	TEXT (NOT NULL)		Business display name
`industry`	TEXT (NOT NULL)		Industry type
`phone_number`	TEXT		Unique phone (E.164)
`agent_name`	TEXT	'AI Assistant'	Voice agent's name
`agent_personality`	JSONB		Tone and personality settings
`voice_config`	JSONB		TTS voice ID and settings
`greeting_standard`	TEXT		Business-hours greeting
`greeting_after_hours`	TEXT		After-hours greeting
`greeting_returning`	TEXT		Returning caller greeting
`timezone`	TEXT		IANA timezone
`operating_hours`	JSONB		Weekly schedule
`escalation_enabled`	BOOLEAN	false	Escalation active
`escalation_phone`	TEXT		Live transfer phone

Column	Type	Default	Description
escalation_triggers	TEXT[]		Trigger phrases
features	JSONB		Feature flags
is_active	BOOLEAN	true	Tenant active
subscription_tier	TEXT	‘starter’	starter/professional/enterprise
voice_pipeline	TEXT (NOT NULL)	‘custom’	‘custom’ or ‘livekit’
status	TEXT	‘draft’	draft/active/suspended
setup_step	TEXT		Current wizard step
setup_completed_at	TIMESTAMPTZ		Setup finalized timestamp
custom_instructions	TEXT		Custom agent instructions
location_city	TEXT		Business city
location_address	TEXT		Business street address
assisted_mode	BOOLEAN	false	Require human confirmation
max_call_duration_seconds	INTEGER	900	Max call length (seconds)
created_at	TIMESTAMPTZ	NOW()	Created timestamp
updated_at	TIMESTAMPTZ	NOW()	Updated timestamp

Constraints: voice_pipeline CHECK IN (‘custom’, ‘livekit’). phone_number has a UNIQUE index.

Indexes: idx_tenants_phone on phone_number, idx_tenants_active on is_active.

8.2 calls

Records of all phone calls handled by the system.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
vapi_call_id	VARCHAR(100)		External call SID
caller_phone	VARCHAR(20)		Caller phone number
caller_name	VARCHAR(255)		Caller name
direction	VARCHAR(20)	‘inbound’	inbound or outbound
status	VARCHAR(20) (NOT NULL)		See CHECK below
started_at	TIMESTAMPTZ		Call start time
ended_at	TIMESTAMPTZ		Call end time
duration_seconds	INTEGER		Duration in seconds
ended_reason	TEXT		Why the call ended
outcome_type	VARCHAR(50)	‘inquiry’	See CHECK below
outcome_success	BOOLEAN	true	Whether outcome succeeded
transcript	TEXT		Full transcript
summary	TEXT		AI-generated summary
sentiment_score	DECIMAL		Sentiment (0.0-1.0)
intents_detected	TEXT[]		Detected intents array
recording_url	TEXT		Recording URL
cost_cents	INTEGER		Cost in cents
contact_id	UUID		FK -> contacts(id)
created_at	TIMESTAMPTZ	NOW()	Created timestamp

Column	Type	Default	Description
updated_at	TIMESTAMPTZ	NOW()	Updated timestamp

CHECK constraints: status IN (ringing, connected, completed, failed, missed). outcome_type IN (booking, inquiry, support, escalation, hangup).

Indexes: idx_calls_tenant_id, idx_calls_vapi_call_id, idx_calls_caller_phone, idx_calls_created_a DESC, idx_calls_status, idx_calls_outcome_type.

8.3 bookings

Appointments, reservations, and orders.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
customer_name	VARCHAR(255)		Customer name
customer_phone	VARCHAR(20)		Customer phone
customer_email	VARCHAR(255)		Customer email
booking_type	VARCHAR(100)	'general'	general, pickup, delivery
booking_date	DATE (NOT NULL)		Date of booking
booking_time	TIME (NOT NULL)		Time of booking
duration_minutes	INTEGER	30	Expected duration
status	VARCHAR(50) (NOT NULL)	'pending'	See CHECK below
confirmation_code	VARCHAR(20)		Unique confirmation code
notes	TEXT		Booking notes
reminder_sent	BOOLEAN	false	Reminder sent flag
source	VARCHAR(50)	'call'	call, web, or manual
call_id	UUID		FK -> calls(id)
resource_id	UUID		FK -> resources(id)
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

CHECK constraints: status IN (pending, confirmed, completed, cancelled, no_show).

Indexes: idx_bookings_tenant_id, idx_bookings_date, idx_bookings_status, idx_bookings_confirmatio

8.4 contacts

CRM contact records with 40+ columns.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
name	VARCHAR(255)		Full name
phone	VARCHAR(20)		Primary phone (E.164)
email	VARCHAR(255)		Primary email
company	VARCHAR(255)		Company name

Column	Type	Default	Description
status	VARCHAR(50)	‘active’	active/inactive/dnc/vip
lead_status	VARCHAR(50)		new/contacted/qualified/etc.
source	VARCHAR(50)		How contact was created
tags	TEXT[]	‘{}’	Segmentation tags
notes_count	INTEGER	0	Cached notes count
total_calls	INTEGER	0	Cached call count
total_bookings	INTEGER	0	Cached booking count
last_call_at	TIMESTAMPTZ		Last call timestamp
last_booking_at	TIMESTAMPTZ		Last booking timestamp
engagement_score	INTEGER	0	Score (0-100)
lifetime_value_cents	INTEGER	0	Total revenue (cents)
preferred_contact_method	VARCHAR(20)		phone/email/sms
preferred_time	VARCHAR(20)		Preferred contact time
language	VARCHAR(10)	‘en’	Preferred language
address_line1	VARCHAR(255)		Street address
address_line2	VARCHAR(255)		Apt/Suite
address_city	VARCHAR(100)		City
address_state	VARCHAR(50)		State
address_zip	VARCHAR(20)		ZIP code
date_of_birth	DATE		Date of birth
custom_fields	JSONB		Custom key-value data
normalized_phone	VARCHAR(20)		Normalized phone format
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Indexes: idx_contacts_tenant_id, idx_contacts_phone, idx_contacts_email, idx_contacts_normalized_
idx_contacts_status, idx_contacts_lead_status, idx_contacts_tags (GIN), idx_contacts_engagement_sc
DESC.

Functions: normalize_phone(text) strips all non-digit characters. update_contact_metrics() trigger updates cached counts.

8.5 contact_notes

Notes attached to contacts.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
contact_id	UUID (NOT NULL)		FK -> contacts(id) CASCADE
note_type	VARCHAR(50)	‘general’	See types below
content	TEXT (NOT NULL)		Note content
call_id	UUID		FK -> calls(id)
is_pinned	BOOLEAN	false	Pinned to top
is_private	BOOLEAN	false	Visible only to creator
created_by	VARCHAR(255)		Creator user/system
created_by_name	VARCHAR(255)		Creator display name
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Note types: general, preference, complaint, compliment, follow_up, internal.

8.6 contact_activity

Activity log for contacts (calls, bookings, status changes, notes).

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
contact_id	UUID (NOT NULL)		FK -> contacts(id)
activity_type	VARCHAR(50) (NOT NULL)		Type of activity
description	TEXT		Activity description
metadata	JSONB		Additional activity data
created_at	TIMESTAMPTZ	NOW()	

8.7 resources

Staff, rooms, and equipment that can be booked.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
name	VARCHAR(255) (NOT NULL)		Resource name
type	VARCHAR(50) (NOT NULL)		staff, room, equipment
description	TEXT		Resource description
is_active	BOOLEAN	true	Active flag
is_bookable	BOOLEAN	true	Bookable flag
display_order	INTEGER	0	Sort order
metadata	JSONB		Additional data
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

8.8 availability_templates

Recurring availability patterns.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
resource_id	UUID		FK -> resources(id)
day_of_week	INTEGER (NOT NULL)		0=Mon through 6=Sun
start_time	TIME (NOT NULL)		Slot start time
end_time	TIME (NOT NULL)		Slot end time
slot_duration_minutes	INTEGER	30	Duration per slot
is_active	BOOLEAN	true	Active flag
created_at	TIMESTAMPTZ	NOW()	

8.9 availability_slots

Individual bookable time slots (generated from templates or manually created).

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
resource_id	UUID		FK -> resources(id)
slot_date	DATE (NOT NULL)		Date of the slot
start_time	TIME (NOT NULL)		Start time
end_time	TIME (NOT NULL)		End time
is_available	BOOLEAN	true	Slot available
is_blocked	BOOLEAN	false	Manually blocked
booking_id	UUID		FK -> bookings(id)
created_at	TIMESTAMPTZ	NOW()	

8.10 notification_templates

Templates for SMS and email notifications.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
name	VARCHAR(100) (NOT NULL)		Template identifier
channel	VARCHAR(20) (NOT NULL)		‘sms’ or ‘email’
subject	TEXT		Email subject line
body	TEXT (NOT NULL)		Body with {{variables}}
variables	TEXT[]		Supported variables
is_active	BOOLEAN	true	
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

8.11 notifications

Sent and queued notification records.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
template_id	UUID		FK -> notification_templates
channel	VARCHAR(20) (NOT NULL)		‘sms’ or ‘email’
recipient_phone	VARCHAR(20)		SMS recipient
recipient_email	VARCHAR(255)		Email recipient
subject	TEXT		Rendered email subject
body	TEXT (NOT NULL)		Rendered message body
status	VARCHAR(20) (NOT NULL)	‘queued’	queued/sending/sent/failed

Column	Type	Default	Description
error_message	TEXT		Error details if failed
sent_at	TIMESTAMPTZ		When sent
retry_count	INTEGER	0	Retry attempts
metadata	JSONB		Additional send data
created_at	TIMESTAMPTZ	NOW()	

8.12 notification_preferences

Per-tenant notification preferences.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL, UNIQUE)		FK -> tenants(id)
booking_confirmation	BOOLEAN	true	Send confirmations
booking_reminder_24h	BOOLEAN	true	Send 24h reminders
booking_reminder_1h	BOOLEAN	true	Send 1h reminders
booking_cancelled	BOOLEAN	true	Send cancellations
missed_call_followup	BOOLEAN	true	Missed call follow-up
review_request	BOOLEAN	false	Send review requests
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

8.13 audit_logs

System audit trail for tracking changes.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
user_id	UUID		User who acted
action	VARCHAR(100) (NOT NULL)		create, update, delete, etc.
entity_type	VARCHAR(100) (NOT NULL)		Table/entity name
entity_id	UUID		Affected record ID
old_values	JSONB		Previous values
new_values	JSONB		New values
ip_address	VARCHAR(45)		Client IP address
user_agent	TEXT		Client user agent
created_at	TIMESTAMPTZ	NOW()	

Indexes: idx_audit_logs_tenant, idx_audit_logs_entity, idx_audit_logs_action, idx_audit_logs_crea
DESC.

8.14 tenant_members

User-tenant membership with role-based access.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
user_id	UUID (NOT NULL)		Supabase auth user ID
role	VARCHAR(20) (NOT NULL)	'member'	owner/admin/member/readonly
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Constraints: UNIQUE on (tenant_id, user_id).

8.15 voicemails

Voicemail recordings when agent is unavailable.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
call_sid	VARCHAR(100) (NOT NULL)		External call identifier
caller_phone	VARCHAR(20) (NOT NULL)		Caller phone number
caller_name	VARCHAR(255)		Caller name
recording_url	TEXT		URL to recording file
recording_sid	VARCHAR(100)		Recording provider ID
duration_seconds	INTEGER		Recording duration
transcript	TEXT		Transcription text
reason	VARCHAR(50) (NOT NULL)		See CHECK below
status	VARCHAR(50) (NOT NULL)	'pending'	See CHECK below
notes	TEXT		Staff notes
reviewed_by	VARCHAR(255)		Who reviewed it
reviewed_at	TIMESTAMPTZ		When reviewed
callback_scheduled_at	TIMESTAMPTZ		Callback scheduled time
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

CHECK constraints: reason IN (after_hours, max_retries, no_agent_available, caller_requested, call_failed). status IN (pending, reviewed, callback_scheduled, resolved).

8.16 conversation_logs

Structured conversation data for LLM training and fine-tuning.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE

Column	Type	Default	Description
call_id	UUID		FK -> calls(id) SET NULL
session_id	TEXT (NOT NULL)		Session identifier
industry	TEXT		Industry type
scenario_type	TEXT		booking/inquiry/support/etc.
language	TEXT	'en'	Conversation language
messages	JSONB (NOT NULL)	'[]'	Message array
quality_score	DECIMAL(3,2)		Score 0.00-1.00
is_complete	BOOLEAN	false	Ended naturally
has_tool_calls	BOOLEAN	false	Contains tool calls
has_escalation	BOOLEAN	false	Contains escalation
outcome_success	BOOLEAN		Outcome successful
turn_count	INTEGER	0	Total turns
user_turns	INTEGER	0	User message count
assistant_turns	INTEGER	0	Assistant message count
tool_calls_count	INTEGER	0	Tool call count
total_tokens_estimate	INTEGER	0	Estimated tokens
duration_seconds	INTEGER		Duration in seconds
reviewed	BOOLEAN	false	Has been reviewed
flagged	BOOLEAN	false	Has been flagged
flag_reason	TEXT		Flag reason
tags	TEXT[]	'{}'	Categorization tags
notes	TEXT		Review notes
exported_at	TIMESTAMPTZ		When exported
export_format	TEXT		jsonl/sharegpt/alpaca
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Indexes: GIN indexes on `messages` and `tags`. Partial indexes on `reviewed WHERE false` and `flagged WHERE true`.

View: `training_data_export` pre-filters to reviewed=true, flagged=false, is_complete=true, quality_score>=0.7.

8.17 callback_queue

Queued callbacks for escalated calls.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
caller_phone	VARCHAR(20) (NOT NULL)		Phone to call back
caller_name	VARCHAR(255)		Caller name
reason	TEXT		Reason for callback
status	VARCHAR(20) (NOT NULL)	'pending'	pending/in_progress/etc.
attempts	INTEGER	0	Callback attempts
scheduled_at	TIMESTAMPTZ		When to attempt
completed_at	TIMESTAMPTZ		When completed
created_at	TIMESTAMPTZ	NOW()	

8.18 sms_messages

SMS message log.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
direction	VARCHAR(10) (NOT NULL)		inbound or outbound
from_number	VARCHAR(20) (NOT NULL)		Sender phone
to_number	VARCHAR(20) (NOT NULL)		Recipient phone
body	TEXT (NOT NULL)		Message content
status	VARCHAR(20)		Delivery status
provider_sid	VARCHAR(100)		Twilio message SID
created_at	TIMESTAMPTZ	NOW()	

8.19 deals

Sales pipeline / CRM deals.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
name	TEXT (NOT NULL)		Deal name
description	TEXT		Deal description
company	TEXT		Company name
stage	TEXT (NOT NULL)		Current pipeline stage
amount_cents	INTEGER	0	Deal value in cents
expected_close	DATE		Expected close date
contact_id	UUID		FK -> contacts(id)
call_id	UUID		FK -> calls(id)
source	TEXT	'manual'	call/web/manual/import
sort_index	INTEGER	0	Position within stage
created_by	TEXT		User or 'auto'
archived_at	TIMESTAMPTZ		Soft-delete timestamp
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Indexes: idx_deals_tenant, idx_deals_stage, idx_deals_contact, idx_deals_archived (partial WHERE archived_at IS NULL).

8.20 tasks

Follow-up tasks and action items.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id) CASCADE
title	TEXT (NOT NULL)		Task title

Column	Type	Default	Description
type	TEXT (NOT NULL)	'custom'	follow_up/call_back/etc.
priority	TEXT (NOT NULL)	'medium'	urgent/high/medium/low
status	TEXT (NOT NULL)	'pending'	pending or completed
due_date	DATE		Due date
notes	TEXT		Task notes
contact_id	UUID		FK -> contacts(id)
call_id	UUID		FK -> calls(id)
deal_id	UUID		FK -> deals(id)
source	TEXT	'manual'	manual or auto
created_by	TEXT		User or 'auto'
completed_at	TIMESTAMPTZ		When completed
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Indexes: idx_tasks_tenant, idx_tasks_status, idx_tasks_due_date, idx_tasks_contact, idx_tasks_priority.

8.21 phone_configurations

Phone number setup details.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
phone_number	TEXT		Configured phone number
setup_type	TEXT (NOT NULL)		new/port/forward/sip
status	TEXT (NOT NULL)	'pending'	See statuses below
forwarding_number	TEXT		Call forwarding target
sip_uri	TEXT		SIP URI for VOIP
sip_username	TEXT		SIP auth username
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Status values: active, pending, porting, porting_with_temp, failed.

Indexes: idx_phone_configurations_sip_uri (partial WHERE sip_uri IS NOT NULL AND setup_type='sip').

8.22 port_requests

Phone number port request tracking.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Primary key
tenant_id	UUID (NOT NULL)		FK -> tenants(id)
phone_number	TEXT (NOT NULL)		Number being ported
carrier	TEXT		Current carrier
account_number	TEXT		Carrier account number
pin	TEXT		Account PIN (encrypted)

Column	Type	Default	Description
status	TEXT (NOT NULL)	'draft'	See statuses below
rejection_reason	TEXT		Port rejection reason
estimated_completion	DATE		Expected completion date
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Status values: draft, submitted, pending, approved, rejected, completed.

8.23 Additional Tables

tenant_capabilities: Industry-specific feature toggles per tenant (tenant_id, capability_key, enabled).

tenant_integrations: OAuth integration records (tenant_id, provider, status, access_token, refresh_token, token_expires_at, scopes, metadata).

tenant_promotions: Active promotions (tenant_id, title, description, is_active, start_date, end_date).

escalation_contacts: Escalation contact list (tenant_id, name, phone, email, priority, display_order).

pending_bookings: Booking requests awaiting human confirmation (tenant_id, caller_name, caller_phone, service_type, requested_date, requested_time, status, call_id, notes).

---

9. Configuration Deep-Dive

This section documents every environment variable used by each component.

9.1 API Environment Variables (`lumentra-api/.env.example`)

Database

Variable	Req	Description
DATABASE_URL	YES	PostgreSQL connection string
SUPABASE_URL	YES	Supabase project URL
SUPABASE_ANON_KEY	YES	Supabase anonymous key
SUPABASE_SERVICE_ROLE_KEY	YES	Service role key (server-side)

LLM Providers

Variable	Req	Description
GEMINI_API_KEY	YES	Gemini key (primary chat)
OPENAI_API_KEY	YES	OpenAI key (fallback + voice)
GROQ_API_KEY	NO	Groq key (tertiary fallback)

Voice Pipeline

Variable	Req	Description
SIGNALWIRE_PROJECT_ID	YES	SignalWire project ID

Variable	Req	Description
`SIGNALWIRE_API_TOKEN`	YES	SignalWire API token
`SIGNALWIRE_SPACE_URL`	YES	SignalWire space URL
`DEEPGRAM_API_KEY`	YES	Deepgram API key (STT)
`CARTESIA_API_KEY`	YES	Cartesia API key (TTS)

Notifications

Variable	Req	Description
`TWILIO_ACCOUNT_SID`	YES	Twilio account SID (SMS)
`TWILIO_AUTH_TOKEN`	YES	Twilio auth token
`TWILIO_PHONE_NUMBER`	YES	Twilio sender phone
`RESEND_API_KEY`	NO	Resend key (email, optional)

Server

Variable	Req	Description
`PORT`	NO	Server port (default: 3100)
`HOST`	NO	Bind address (default: 0.0.0.0)
`NODE_ENV`	NO	development or production
`INTERNAL_API_KEY`	YES	Internal API bearer token
`ENCRYPTION_KEY`	NO	AES-256 key for OAuth tokens

OAuth Integration

Variable	Req	Description
`GOOGLE_CLIENT_ID`	NO	Google OAuth client ID
`GOOGLE_CLIENT_SECRET`	NO	Google OAuth client secret
`OUTLOOK_CLIENT_ID`	NO	Microsoft OAuth client ID
`OUTLOOK_CLIENT_SECRET`	NO	Microsoft OAuth secret
`CALENDLY_CLIENT_ID`	NO	Calendly OAuth client ID
`CALENDLY_CLIENT_SECRET`	NO	Calendly OAuth secret

9.2 Dashboard Environment Variables (`lumentra-dashboard/.env.example`)

Variable	Req	Description
`NEXT_PUBLIC_API_URL`	YES	API base URL
`NEXT_PUBLIC_SUPABASE_URL`	YES	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	YES	Supabase anonymous key
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	NO	Stripe publishable key
`STRIPE_SECRET_KEY`	NO	Stripe secret key

9.3 Agent Environment Variables (`lumentra-agent/.env.example`)

Variable	Req	Description
`LIVEKIT_URL`	YES	LiveKit server WebSocket URL
`LIVEKIT_API_KEY`	YES	LiveKit API key

Variable	Req	Description
LIVEKIT_API_SECRET	YES	LiveKit API secret
INTERNAL_API_URL	YES	API internal URL
INTERNAL_API_KEY	YES	Internal API bearer token
DEEPGRAM_API_KEY	YES	Deepgram API key (STT)
OPENAI_API_KEY	YES	OpenAI key (voice LLM)
CARTESIA_API_KEY	YES	Cartesia key (TTS)

9.4 Encryption Configuration

The `ENCRYPTION_KEY` environment variable enables AES-256-GCM encryption for sensitive stored data (OAuth tokens, port request PINs). The encryption service (`lumentra-api/src/services/crypto/encryption.t` provides:

- **Algorithm:** AES-256-GCM with per-value random 16-byte IVs
- **Key Derivation:** scrypt with a fixed salt ("lumentra-encryption-salt") producing a 32-byte key
- **Storage Format:** `enc:<iv_hex>:<auth_tag_hex>:<ciphertext_hex>`
- **Graceful Degradation:** When `ENCRYPTION_KEY` is not set, encrypt/decrypt operations are skipped (values stored in plaintext). This allows development without encryption configured.
- **Functions:** `encrypt(plaintext)`, `decrypt(value)`, `isEncrypted(value)`, `encryptIfNeeded(value)`

---

10. Background Jobs Reference

The job scheduler runs via `node-cron` and is initialized in `lumentra-api/src/jobs/scheduler.ts`. Six background jobs run on fixed schedules.

10.1 Reminder Processing

Schedule: Every 10 minutes (`*/10 * * * *`) **File:** `lumentra-api/src/jobs/reminders.ts` **Function:** `sendDueReminders()`

Sends booking reminders via the notification service:

1. **24-hour reminders:** Finds bookings with `booking_date` tomorrow and `reminder_sent = false`. Sends via `booking_reminder_24h` template.
2. **1-hour reminders:** Finds bookings with `booking_date` today and `booking_time` within the next 60-90 minutes. Sends via `booking_reminder_1h` template.

After sending, marks `reminder_sent = true` on the booking record.

10.2 Callback Processing

Schedule: Every 5 minutes (`*/5 * * * *`) **File:** `lumentra-api/src/jobs/callbacks.ts` **Function:** `processCallbacks()`

Processes the callback queue:

1. Fetches callback_queue entries with `status = 'pending'` and `attempts < 3`
2. Marks each as `status = 'in_progress'`
3. Attempts the callback (currently logs the attempt)
4. On success: marks as `status = 'completed'`
5. On failure: increments `attempts` counter, remains pending for retry

10.3 Notification Queue Processing

Schedule: Every 15 minutes (`*/15 * * * *`) **File:** `lumentra-api/src/services/notifications/notification service.ts` **Function:** `processNotificationQueue()`

Processes queued notifications:

1. Fetches notifications with `status = 'queued'` and `retry_count < 3`
2. For each notification:
   - **SMS:** Sends via Twilio client (`twilio.messages.create`)
   - **Email:** Sends via Resend client (`resend.emails.send`)
3. On success: updates `status = 'sent'` and `sent_at`
4. On failure: increments `retry_count`, logs error message. Uses exponential backoff.

10.4 Engagement Score Calculation

Schedule: Every hour (`0 * * * *`) **File:** `lumentra-api/src/jobs/engagement.ts` **Function:** `recalculateEngagementScores()`

Recalculates engagement scores for all contacts across all tenants:

Score Components (total 100 points):

Recency (max 30 points): Days since last interaction. 0-7d = 30, 8-14d = 25, 15-30d = 20, 31-60d = 10, 61-90d = 5, 90+ = 0.

Frequency (max 30 points): Interactions in last 90 days. 10+ = 30, 7-9 = 25, 4-6 = 20, 2-3 = 15, 1 = 10, 0 = 0.

Completion (max 20 points): Booking completion rate. 100% = 20, 75%+ = 15, 50%+ = 10, 25%+ = 5, under 25% = 0.

Value (max 20 points): Lifetime value. $500+ = 20, $200+ = 15, $100+ = 10, $50+ = 5, under $50 = 0.

Updates the `engagement_score` column on each contact record.

10.5 Availability Slot Generation

Schedule: Daily at midnight (`0 0 * * *`) **File:** `lumentra-api/src/jobs/availability-generator.ts` **Function:** `generateDailySlots()`

Generates availability slots for the next 30 days:

1. For each active tenant with operating hours configured
2. For each day in the next 30 days
3. Creates availability_slots based on the tenant's operating hours schedule
4. Skips days where the tenant is closed
5. Also cleans up slots older than 90 days

10.6 Review Request Sending

Schedule: Daily at 9:00 AM (`0 9 * * *`) **File:** `lumentra-api/src/jobs/review-requests.ts` **Function:** `sendReviewRequests()`

Sends review/feedback requests to customers after completed appointments:

1. Finds bookings with `status = 'completed'` where `completed_at` is between 24 and 48 hours ago
2. Checks that the tenant has `review_request` enabled in notification preferences
3. Sends review request via SMS and/or email using the appropriate template
4. Records that the review request was sent to prevent duplicates

---

11. Webhook and Integration Reference

11.1 SignalWire SIP Webhook

Endpoint: `POST /sip/forward` (no authentication)

Called by SignalWire when an inbound call arrives. Returns TwiML XML that directs the call to the LiveKit SIP endpoint.

11.2 LiveKit Agent Communication

The LiveKit Python agent communicates with the API via three internal endpoints (documented in Section 7.3):

1. `GET /internal/tenants/by-phone/:phone` – Get tenant config at call start
2. `POST /internal/voice-tools/:action` – Execute tools during call
3. `POST /internal/calls/log` – Log call after completion

All authenticated via `INTERNAL_API_KEY` Bearer token.

11.3 Twilio Webhooks

Voicemail Recording Complete: `POST /api/voicemails/callback/complete` Called when a voicemail recording finishes. Receives `RecordingUrl`, `RecordingSid`, `RecordingDuration`.

Voicemail Status: `POST /api/voicemails/callback/status` Called when voicemail delivery status changes.

Voicemail Transcription: `POST /api/voicemails/callback/transcribe` Called when voicemail transcription is complete. Receives `TranscriptionText`.

11.4 OAuth Integration Flow

The OAuth flow for calendar integrations works as follows:

1. **Authorize:** Dashboard calls `GET /api/integrations/:provider/authorize` which returns an `authUrl`
2. **Popup:** Dashboard opens the `authUrl` in a popup window (600x700px)
3. **User Consent:** User logs in and grants permissions at the provider
4. **Callback:** Provider redirects to `GET /api/integrations/:provider/callback` with an authorization code
5. **Token Exchange:** API exchanges the code for access/refresh tokens
6. **Encryption:** Tokens are encrypted using AES-256-GCM before storage
7. **Storage:** Integration record created in `tenant_integrations` with status "active"
8. **Notification:** Callback page renders HTML that posts `{type: "oauth_success"}` to the parent window

Supported OAuth Providers:

Provider	Scopes	Refresh
`google_calendar`	calendar.readonly, events	Yes
`outlook`	Calendars.ReadWrite	Yes
`calendly`	default	Yes

Token Refresh: When an access token expires, the API uses the stored refresh token to obtain a new access token. The `POST /api/integrations/:id/refresh` endpoint can be called manually.

11.5 Post-Call Automation

After every call is logged via `/internal/calls/log`, the post-call automation system runs asynchronously (non-blocking). The automation rules are defined in `lumentra-api/src/services/automation/postcall.ts`:

Rule 1 – Booking Made:

- Trigger: `outcome_type = "booking"`
- Action: Create a deal with stage set to the industry's completed stage, set contact `lead_status` to "converted"

Rule 2 – Escalation:

- Trigger: `outcome_type = "escalation"`
- Action: Create a task with type "call_back", priority "high", due tomorrow

Rule 3 – Missed/Short Call:

- Trigger: `status = "missed"` OR `duration_seconds < 10`
- Action: Create a task with type "call_back", priority "high", due today

Rule 4 – VIP Upgrade:

- Trigger: Contact has 3+ calls AND engagement_score >= 80 AND status is not already "vip"
- Action: Update contact status to "vip"

Rule 5 – First-Time Inquiry:

- Trigger: Contact has <= 1 call AND `outcome_type = "inquiry"`
- Action: Create a deal with the industry's default stage, set contact `lead_status` to "new"

---

12. Permission Matrix

This matrix shows which roles can access which API endpoints.

12.1 Tenant Management

Endpoint	Owner	Admin	Member	RO
`GET /api/tenants`	Yes	Yes	Yes	Yes
`POST /api/tenants`	Yes	Yes	Yes	Yes
`GET /api/tenants/:id`	Yes	Yes	Yes	Yes
`PUT /api/tenants/:id`	Yes	Yes	No	No
`PATCH .../phone`	Yes	Yes	No	No
`GET .../members`	Yes	Yes	Yes	Yes
`POST .../members`	Yes	Yes	No	No
`PATCH .../members/:userId`	Yes	Yes	No	No
`DELETE .../members/:userId`	Yes	Yes	No	No

12.2 CRM Data (Calls, Contacts, Bookings)

Endpoint Pattern	Owner	Admin	Member	RO
`GET /api/calls/*`	Yes	Yes	Yes	Yes
`GET /api/contacts/*`	Yes	Yes	Yes	Yes
`POST /api/contacts`	Yes	Yes	Yes	No
`PUT /api/contacts/:id`	Yes	Yes	Yes	No
`DELETE /api/contacts/:id`	Yes	Yes	No	No
`POST .../contacts/import`	Yes	Yes	No	No
`POST .../contacts/merge`	Yes	Yes	No	No
`GET /api/bookings/*`	Yes	Yes	Yes	Yes
`POST /api/bookings`	Yes	Yes	Yes	No
`PUT /api/bookings/:id`	Yes	Yes	Yes	No
`PATCH .../bookings/:id/*`	Yes	Yes	Yes	No

12.3 Pipeline (Deals & Tasks)

Endpoint Pattern	Owner	Admin	Member	RO
`GET /api/deals/*`	Yes	Yes	Yes	Yes
`POST /api/deals`	Yes	Yes	Yes	No
`PUT /api/deals/:id`	Yes	Yes	Yes	No
`PATCH .../deals/:id/stage`	Yes	Yes	Yes	No
`DELETE /api/deals/:id`	Yes	Yes	No	No
`GET /api/tasks/*`	Yes	Yes	Yes	Yes
`POST /api/tasks`	Yes	Yes	Yes	No
`PUT /api/tasks/:id`	Yes	Yes	Yes	No
`PATCH .../tasks/:id/complete`	Yes	Yes	Yes	No
`DELETE /api/tasks/:id`	Yes	Yes	No	No

12.4 Escalation

Endpoint Pattern	Owner	Admin	Member	RO
`GET .../escalation/queue`	Yes	Yes	Yes	Yes

Endpoint Pattern	Owner	Admin	Member	RO
`POST .../queue/:id/take`	Yes	Yes	Yes	No
`POST .../queue/:id/resolve`	Yes	Yes	Yes	No
`POST .../schedule-callback`	Yes	Yes	Yes	No
`GET .../escalation/contacts`	Yes	Yes	Yes	Yes
`POST .../escalation/contacts`	Yes	Yes	No	No
`PUT .../escalation/contacts/:id`	Yes	Yes	No	No
`DELETE .../contacts/:id`	Yes	Yes	No	No
`PUT .../escalation/triggers`	Yes	Yes	No	No

12.5 Settings & Configuration

Endpoint Pattern	Owner	Admin	Member	RO
`GET .../notifications/*`	Yes	Yes	Yes	Yes
`POST .../notifications/send`	Yes	Yes	No	No
`.../notifications/templates/*`	Yes	Yes	No	No
`PUT .../notif. preferences`	Yes	Yes	No	No
`GET .../capabilities/*`	Yes	Yes	Yes	Yes
`PUT .../capabilities`	Yes	Yes	No	No
`.../promotions/*` (write)	Yes	Yes	No	No
`GET .../integrations/*`	Yes	Yes	Yes	Yes
`DELETE .../integrations/:id`	Yes	Yes	No	No
`.../phone/*` (write)	Yes	Yes	No	No

12.6 Rate Limits

Rate limits are applied at the middleware level:

Preset	Requests	Window	Applied To
`rateLimit`	60	1 min	Default for most endpoints
`strictRateLimit`	10	1 min	Sensitive write ops
`criticalRateLimit`	5	1 hour	Account-level ops
`readRateLimit`	120	1 min	Read-heavy endpoints
`tenantRateLimit`	300	1 min	Per-tenant aggregate

Preset	Requests	Window	Applied To

Rate limiting uses an in-memory store keyed by IP address (or tenant ID for tenant-scoped limits). When a limit is exceeded, the API returns `429 Too Many Requests` with a `Retry-After` header.

---

13. Audit Trail Documentation

13.1 Audit Log Table

The `audit_logs` table (Section 8.13) records significant system events. Each entry captures:

- **Who:** `user_id` of the authenticated user
- **What:** `action` (create, update, delete) and `entity_type` (tenants, contacts, bookings, etc.)
- **Which:** `entity_id` of the affected record
- **When:** `created_at` timestamp
- **Changes:** `old_values` and `new_values` JSONB columns showing before/after state
- **Context:** `ip_address` and `user_agent` of the client

13.2 Audited Actions

The following actions generate audit log entries:

Action	Entity Type	Description
`create`	tenants	New tenant created
`update`	tenants	Tenant settings changed
`create`	contacts	New contact created
`update`	contacts	Contact info updated
`delete`	contacts	Contact soft-deleted
`merge`	contacts	Two contacts merged
`import`	contacts	Bulk contact import
`create`	bookings	New booking created
`update`	bookings	Booking modified
`status_change`	bookings	Status transition
`create`	deals	New deal created
`update`	deals	Deal modified
`stage_change`	deals	Deal moved between stages
`archive`	deals	Deal archived
`create`	tasks	New task created
`complete`	tasks	Task marked complete
`member_add`	tenant_members	New member added
`member_update`	tenant_members	Member role changed
`member_remove`	tenant_members	Member removed
`integration_connect`	tenant_integrations	OAuth connected
`integration_disconnect`	tenant_integrations	Integration removed

13.3 Querying Audit Logs

Audit logs are tenant-scoped and protected by RLS. To query:

```sql
SELECT action, entity_type, entity_id, old_values, new_values, created_at
FROM audit_logs
WHERE tenant_id = '<tenant-uuid>'
ORDER BY created_at DESC
LIMIT 100;
```

---

14. Multi-Tenancy Deep-Dive

Lumentra implements multi-tenancy at three levels: database (Row Level Security), application cache, and API middleware.

14.1 Database Level – Row Level Security (RLS)

All data tables have RLS enabled with policies that restrict access to the owning tenant. The RLS policies work in two modes:

Mode 1 – Session Variable (for direct DB queries):

```
CREATE POLICY "Tenants can view own data"
    ON contacts FOR SELECT
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

Before executing queries, the API sets the session variable:

```
SET app.current_tenant_id = '<tenant-uuid>';
```

Mode 2 – Membership Check (for auth-protected tables):

```
CREATE POLICY "Members can view tenant data"
    ON calls FOR SELECT
    USING (
      tenant_id IN (
        SELECT tenant_id FROM tenant_members
        WHERE user_id = auth.uid()
      )
    );
```

Tables with RLS enabled:

tenants, calls, bookings, contacts, contact_notes, contact_activity, resources, availability_templates, availability_slots, notification_templates, notifications, notification_preferences, audit_logs, voicemails, conversation_logs, deals, tasks, callback_queue, sms_messages, phone_configurations, port_requests, tenant_capabilities, tenant_integrations, tenant_promotions, escalation_contacts, pending_bookings

14.2 Application Cache Level

The tenant cache (`lumentra-api/src/services/database/tenant-cache.ts`) provides sub-millisecond tenant lookups for the voice pipeline:

Architecture:

- In-memory `Map` objects keyed by phone number and tenant ID
- O(1) lookup time for incoming calls
- Refreshed every 5 minutes from the database
- Manual invalidation on tenant updates

Cache Operations:

- `getTenantByPhone(phone)` – Primary lookup for incoming calls
- `getTenantByPhoneWithFallback(phone)` – Checks cache first, falls back to SIP URI lookup in DB
- `getTenantById(id)` – Lookup by tenant UUID
- `invalidateTenant(id)` – Removes tenant from cache (forces re-fetch)
- `refreshCache()` – Reloads all tenants from database

Why In-Memory: The voice pipeline requires sub-50ms latency for webhook responses. Database queries add 5-20ms per lookup. With the cache, lookups are < 0.1ms.

Cache Refresh Cycle:

1. On startup: loads all active tenants from `SELECT * FROM tenants WHERE is_active = true`
2. Every 5 minutes: full refresh from database
3. On tenant update: immediate cache invalidation for the updated tenant
4. On phone number change: invalidate old phone mapping, create new one

14.3 API Middleware Level

The authentication middleware enforces tenant isolation at the API level:

Step 1 – JWT Verification:

```
Authorization: Bearer <supabase_jwt>
```

The middleware calls `supabase.auth.getUser(token)` to verify the JWT and extract the user ID.

Step 2 – Tenant Context:

```
X-Tenant-ID: <tenant-uuid>
```

The middleware checks that the authenticated user has a record in `tenant_members` for the specified tenant.

Step 3 – Role Extraction:

The user's role (owner, admin, member, readonly) is extracted from the tenant_members record and attached to the request context.

Step 4 – Tenant Scoping:

All database queries include `WHERE tenant_id = <tenant-uuid>` to ensure data isolation. This is redundant with RLS but provides defense-in-depth.

14.4 Cross-Tenant Data Isolation

Data never leaks between tenants because of the layered approach:

1. **RLS** prevents any SQL query from accessing another tenant's data, even if the application has a bug
2. **Middleware** validates tenant membership before the route handler runs
3. **Query helpers** automatically inject `tenant_id` into INSERT statements
4. **Cache** is keyed by phone number, which is unique per tenant
5. **Internal API** authenticates via a separate `INTERNAL_API_KEY`, not user JWTs

---

15. Notification Templates

15.1 Built-in Templates

The notification service includes five built-in templates that are seeded automatically:

booking_confirmation (SMS)

```
Hi {{customer_name}}, your {{booking_type}} at {{business_name}} is confirmed for {{booking_date
```

Variables: customer_name, booking_type, business_name, booking_date, booking_time, confirmation_code

booking_reminder_24h (SMS)

`Reminder: You have a {{booking_type}} at {{business_name}} tomorrow at {{booking_time}}. Confirma`

Variables: customer_name, booking_type, business_name, booking_date, booking_time, confirmation_code

booking_reminder_1h (SMS)

`{{customer_name}}, your {{booking_type}} at {{business_name}} is in 1 hour at {{booking_time}}. S`

Variables: customer_name, booking_type, business_name, booking_time

booking_cancelled (SMS)

`Hi {{customer_name}}, your {{booking_type}} at {{business_name}} on {{booking_date}} at {{bookin`

Variables: customer_name, booking_type, business_name, booking_date, booking_time

missed_call_followup (SMS)

`Hi, we noticed you called {{business_name}} and we missed you. We'd love to help! Call us back or r`

Variables: business_name

15.2 Template Rendering

Templates use `{{variable_name}}` mustache-style interpolation. The rendering engine replaces all `{{variable}}` placeholders with the provided values:

```
// Internal rendering logic
function renderTemplate(body: string, variables: Record<string, string>): string {
  let rendered = body;
  for (const [key, value] of Object.entries(variables)) {
    rendered = rendered.replaceAll(`{{${key}}}`, value);
  }
  return rendered;
}
```

15.3 Notification Channels

SMS (Twilio):

- Provider: Twilio Programmable SMS
- Sender: Configured via `TWILIO_PHONE_NUMBER` env var
- Rate limit: Twilio's default limits apply
- Cost: Standard Twilio SMS rates

Email (Resend):

- Provider: Resend
- API Key: `RESEND_API_KEY` env var
- Optional: If not configured, email notifications are skipped

15.4 Notification Preferences

Each tenant can enable/disable specific notification types via the `notification_preferences` table:

Preference	Default	Description
`booking_confirmation`	true	Confirm after booking

Preference	Default	Description
`booking_reminder_24h`	true	Remind 24 hours before
`booking_reminder_1h`	true	Remind 1 hour before
`booking_cancelled`	true	Notice on cancellation
`missed_call_followup`	true	Follow up missed calls
`review_request`	false	Review request after visit

---

16. Managing Voice Agents

16.1 Voice Agent Architecture

Each tenant's voice agent is configured through three components:

1. **System Prompt:** Generated by `buildSystemPrompt()` in `lumentra-api/src/services/gemini/chat.ts`. Includes business name, industry context, agent personality, operating hours, location, custom instructions, and escalation phone.
2. **Voice Configuration:** TTS voice selection stored in `voice_config` JSONB column. Six voices available (Sarah, Emma, Maya, James, Alex, David).
3. **Agent Personality:** Stored in `agent_personality` JSONB column. Three personality types (professional, friendly, efficient) that control greeting style and conversation tone.

16.2 Voice Tool Functions

The voice agent can call six tools during a conversation:

- **`check_availability`** (`executeCheckAvailability`): Checks available time slots for a date. Returns up to 3 slots formatted for voice.
- **`create_booking`** (`executeCreateBooking`): Creates a confirmed booking with a 6-char alphanumeric confirmation code.
- **`create_order`** (`executeCreateOrder`): Creates a pickup/delivery order. Validates name, order type, items, and delivery address.
- **`transfer_to_human`** (`executeTransferToHuman`): Initiates transfer to escalation phone via SIP REFER. Updates outcome to "escalation".
- **`end_call`** (`executeEndCall`): Signals call completion. LiveKit agent handles session shutdown.
- **`log_note`** (`executeLogNote`): Saves a note to the caller's contact record. Finds or creates contact by phone.

16.3 Tool Input Validation

The `create_order` tool performs strict input validation to prevent hallucinated or placeholder values:

- **Invalid values rejected:** "unknown", "not provided", "n/a", "none", "undefined", "null", empty string
- **Required fields:** customer_name, order_type (pickup/delivery), items
- **Conditional:** delivery_address required when order_type is "delivery"
- **Fallback:** If customer_phone is missing, uses the caller's phone from the call context

16.4 Greeting Configuration

Three types of greetings are supported:

- **Standard:** Used during business hours
- **After Hours:** Used outside operating hours
- **Returning:** Used when the caller has called before (contact exists in the system)

If custom greetings are not set, the system prompt builder generates appropriate greetings based on the agent personality and business name.

---

17. Phone Number Management

17.1 Phone Setup Types

Four phone setup methods are available:

Type	Description	Status Flow
`new`	Provision new SignalWire number	pending -> active
`port`	Transfer existing number	pending -> porting -> active
`forward`	Forward to a Lumentra line	pending -> active
`sip`	Direct SIP trunk (VOIP/PBX)	pending -> active

Port requests may also use `porting_with_temp` status when a temporary number is provisioned during porting.

17.2 Number Porting Process

1. Submit port request with current carrier, account number, and PIN
2. PIN is encrypted using AES-256-GCM before storage
3. Port request status progresses: draft -> submitted -> pending -> approved -> completed (or rejected)
4. Optional: A temporary number can be provisioned while porting is in progress (`porting_with_temp` status)
5. When port completes, the tenant's phone number is updated and the cache is invalidated

17.3 Call Forwarding Setup

1. A Lumentra forwarding number is provisioned
2. The user configures call forwarding on their carrier using carrier-specific instructions
3. Supported carriers with step-by-step instructions: AT&T (*72), Verizon (*71), T-Mobile (Settings), Sprint (*72)
4. Verification endpoint confirms forwarding is working

17.4 SIP Configuration

For businesses with existing VOIP/PBX systems: - SIP URI and username are stored in `phone_configurations` - The tenant cache performs SIP URI lookups with database fallback - Index: `idx_phone_configurations_sip_uri` for efficient lookup

---

18. Escalation Management

18.1 Escalation Triggers

Escalation can be triggered by:

1. **Keyword triggers:** Phrases configured in `escalation_triggers` (e.g., "speak to manager", "emergency", "urgent")
2. **Agent judgment:** The LLM decides the caller needs human assistance

3. **Tool failure:** When a tool call fails and the agent cannot recover
4. **Caller frustration:** When sentiment analysis detects negative sentiment

18.2 Escalation Flow

1. Voice agent detects an escalation trigger
2. Agent calls `transfer_to_human` tool with reason
3. API updates call outcome to "escalation"
4. If `escalation_phone` is configured: LiveKit agent performs SIP REFER transfer
5. If no escalation phone: Agent apologizes and offers callback
6. Escalation is logged in the escalation queue
7. Post-call automation creates a high-priority callback task

18.3 Escalation Queue

The escalation queue provides real-time management:

- **Statuses:** waiting, in-progress, callback-scheduled, resolved
- **Priorities:** urgent, high, normal, low
- **Actions:** Take call (assign to yourself), schedule callback, resolve
- **Metrics:** Waiting count, high priority count, average wait time, resolved today

18.4 Escalation Contacts

Multiple escalation contacts can be configured with priority ordering: - Each contact has name, phone, email, and priority level - Contacts are tried in priority order during live transfers - Reordering is supported via the API

---

19. CRM and Automation

19.1 Contact Lifecycle

1. **Auto-Creation:** Contacts are automatically created when a new caller calls (via `findOrCreateByPhone`)
2. **Enrichment:** Name, email, and preferences are added during calls and manual edits
3. **Engagement Tracking:** Engagement score updated hourly based on recency, frequency, completion, and value
4. **VIP Upgrade:** Contacts with 3+ calls and engagement score >= 80 are automatically upgraded to VIP status
5. **Lead Status:** Tracks the contact through: new -> contacted -> qualified -> converted | lost

19.2 Deal Pipeline

Deals represent business opportunities and use industry-specific stages:

- **Auto-Creation:** First-time callers with inquiry outcome automatically create a new deal
- **Booking Conversion:** Booking outcomes automatically create a won deal and set lead status to "converted"
- **Kanban Board:** Visual pipeline with drag-and-drop stage changes
- **Stage Validation:** API validates that the target stage exists in the tenant's industry pipeline

19.3 Task Management

Tasks track follow-up actions:

- **Auto-Creation:** Escalated and missed calls automatically create callback tasks

- **Priority System:** urgent (red), high (amber), medium (blue), low (gray)
- **Due Date Tracking:** Overdue tasks are highlighted and can be filtered
- **Industry Types:** Universal types (follow_up, call_back, email, meeting, review, custom) plus industry-specific types

19.4 Contact Import and Export

Import:

- Up to 10,000 records per request
- Required: at least one of phone or email
- Optional: name, company, tags, status, custom_fields
- Duplicate detection by phone number

Export:

- JSON format: array of contact objects
- CSV format: flat file with standard columns
- All contacts for the tenant are included

---

20. Monitoring and Health

20.1 Health Endpoints

`GET /health`: Full system health check - Database connectivity test - Cache statistics (tenant count, last refresh) - Server uptime

`GET /health/ping`: Lightweight liveness probe - Returns `{ "pong": true }` - Use for load balancer health checks

20.2 Dashboard Metrics

The dashboard home page provides real-time monitoring:

- **System Health Panel:** Database status, cache stats, API latency
- **Activity Log:** Real-time feed of calls, bookings, escalations
- **Waveform:** Live call visualization

20.3 Key Metrics to Monitor

Metric	Source	Warning
DB connectivity	`/health`	Any “disconnected”
Cache tenant count	`/health`	0 tenants (empty)
Cache refresh age	`/health`	> 10 min since refresh
API response time	App logs	> 500ms average
Call failure rate	`/api/calls/stats`	> 5% failed
Escalation depth	`/api/escalation/queue`	> 10 waiting
Notification fails	Service logs	> 3 consecutive

---

21. Server and Container Management

21.1 Deployment Architecture

- **Coolify** (port 8000): Manages lumentra-api and lumentra-dashboard containers

- **LiveKit Stack** (`/opt/livekit/docker-compose.yml`): Managed separately, not via Coolify
  - All services use `network_mode: host`
  - Agent env vars loaded via `env_file: .env`
  - Agent reaches API at `http://10.0.1.5:3100` (Coolify network IP)

21.2 Container Management

API Container (via Coolify):

```
# Deploy via Coolify artisan tinker
docker exec coolify php artisan tinker --execute="..."
# API UUID: scog8ocs4884cos8gscw0kss
# Dashboard UUID: hc44wc84swwo80s8k4gw88oo
```

LiveKit Stack:

```
cd /opt/livekit
docker compose up -d        # Start all services
docker compose restart agent # Restart agent only
docker compose logs -f agent # View agent logs
```

21.3 Port Requirements

Port	Protocol	Service
22	TCP	SSH
80	TCP	HTTP (Coolify)
443	TCP	HTTPS (Coolify)
3100	TCP	Lumentra API
5060	UDP/TCP	SIP (LiveKit SIP)
7880-7881	TCP	LiveKit Server
8000	TCP	Coolify admin
10000-20000	UDP	RTP media
50000-60000	UDP	WebRTC ICE

---

22. Security

22.1 Authentication Layers

1. **User Authentication:** Supabase JWT tokens (RS256 signed)
2. **Internal API:** INTERNAL_API_KEY Bearer token (shared secret between API and agent)
3. **OAuth Tokens:** AES-256-GCM encrypted at rest with per-value random IVs

22.2 Data Protection

- **Row Level Security:** All tables have RLS enabled with tenant-scoped policies
- **Token Encryption:** OAuth tokens encrypted before database storage
- **Input Validation:** Zod schemas validate all API input
- **Rate Limiting:** Per-IP and per-tenant rate limits prevent abuse
- **CORS:** Configured to allow only the dashboard origin

22.3 Sensitive Data Handling

- Port request PINs are encrypted using `encryptIfNeeded()` before storage
- OAuth access and refresh tokens are encrypted

- API keys and secrets are stored in environment variables only
- Database credentials use connection strings (not hardcoded)
- Supabase service role key is server-side only (never exposed to client)

22.4 Rate Limiting Details

The rate limiter uses an in-memory store with sliding window:

```
interface RateLimitConfig {
  windowMs: number;  // Time window in milliseconds
  max: number;       // Maximum requests per window
}
```

When exceeded, returns:

```
{
  "error": "Too many requests",
  "retryAfter": 45
}
```

HTTP Status: 429 with `Retry-After` header.

---

23. Troubleshooting

23.1 Voice Pipeline Issues

Problem: Calls not being answered

1. Check LiveKit agent is running: `docker compose logs agent`
2. Verify tenant exists and has correct phone number
3. Check tenant cache is populated: `GET /health` cache stats
4. Verify INTERNAL_API_KEY matches between agent and API
5. Check SIP trunk is registered: LiveKit SIP logs

Problem: Agent cannot reach API

1. Verify INTERNAL_API_URL in agent .env points to correct IP (10.0.1.5:3100)
2. Check Coolify network connectivity
3. Test: `curl -H "Authorization: Bearer <key>" http://10.0.1.5:3100/internal/tenants/by-phone/+15551234567`

Problem: Poor voice quality

1. Check Deepgram API key is valid
2. Check Cartesia API key is valid
3. Verify TTS speed settings (default: 0.95)
4. Check server CPU usage (voice processing is CPU-intensive)

23.2 Authentication Issues

Problem: 401 Unauthorized

- JWT token expired or invalid
- Missing Authorization header
- Supabase project URL mismatch

Problem: 403 Forbidden

- User does not have required role
- INTERNAL_API_KEY mismatch (for internal routes)
- User not a member of the specified tenant

Problem: Missing X-Tenant-ID

- Header not included in request
- Tenant ID format is not a valid UUID

23.3 Database Issues

Problem: RLS policy blocking queries

- Ensure `app.current_tenant_id` session variable is set before queries
- Check that the user's JWT maps to a valid tenant_members record
- Verify RLS policies are not overly restrictive for the operation

Problem: Slow queries

- Check index usage with `EXPLAIN ANALYZE`
- Most common slow queries: contact search without phone normalization, call listing without date range
- Consider adding date range filters to reduce result sets

23.4 Notification Issues

Problem: SMS not sending

1. Verify Twilio credentials (TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
2. Check TWILIO_PHONE_NUMBER is a valid Twilio number
3. Check notification queue: notifications with status "failed" have error_message
4. Verify notification preferences are enabled for the event type

Problem: Email not sending

1. Verify RESEND_API_KEY is set
2. Check Resend dashboard for delivery status
3. Email notifications are optional – if RESEND_API_KEY is not set, they are silently skipped

23.5 Cache Issues

Problem: Tenant changes not reflected

1. Tenant cache refreshes every 5 minutes
2. For immediate updates, the API invalidates the cache on tenant update
3. If cache is stale, restart the API container to force a full reload

Problem: Incoming calls going to wrong tenant

1. Check phone number uniqueness across tenants
2. Verify the cache has the correct phone-to-tenant mapping
3. Use `GET /internal/tenants/by-phone/:phone` to test the lookup

23.6 Integration Issues

Problem: OAuth callback fails

1. Verify OAuth client ID and secret are correct
2. Check that callback URL matches the configured redirect URI in the provider
3. Verify ENCRYPTION_KEY is set (tokens cannot be stored without it in production)
4. Check browser console for popup blocker messages

Problem: Integration shows "Expired"

1. Access token has expired and refresh failed
2. Try `POST /api/integrations/:id/refresh` to manually refresh
3. If refresh fails, disconnect and reconnect the integration

23.7 Background Job Issues

Problem: Reminders not sending

1. Check that the job scheduler started (look for "[SCHEDULER]" in API logs)
2. Verify notification preferences have `booking_reminder_24h` and `booking_reminder_1h` enabled
3. Check that bookings have `reminder_sent = false`
4. Verify the notification queue is being processed (check for queued notifications)

Problem: Engagement scores not updating

1. Job runs hourly – wait for the next cycle
2. Check for errors in the "[ENGAGEMENT]" log prefix
3. Verify contacts have associated calls and bookings for score calculation

---

End of Lumentra Administration Guide v2.0